Introduction aux statistiques avec R

Bioinformatique I (BIF-7900)

Antoine Bodein

5 novembre 2026



1

INTRODUCTION

Objectifs

Deux somnifères, 10 patients, les heures de sommeil gagnées avec chacun (`sleep`, W. S. Gosset, 1908) :

```
t.test(extra ~ group, data = sleep)$p.value
#> [1] 0.07939414
```

« Pas de différence entre les somnifères. »

```
t.test(sommeil$som_2, sommeil$som_1,
       paired = TRUE)$p.value
#> [1] 0.00283289
```

« Le somnifère 2 fait mieux dormir. »

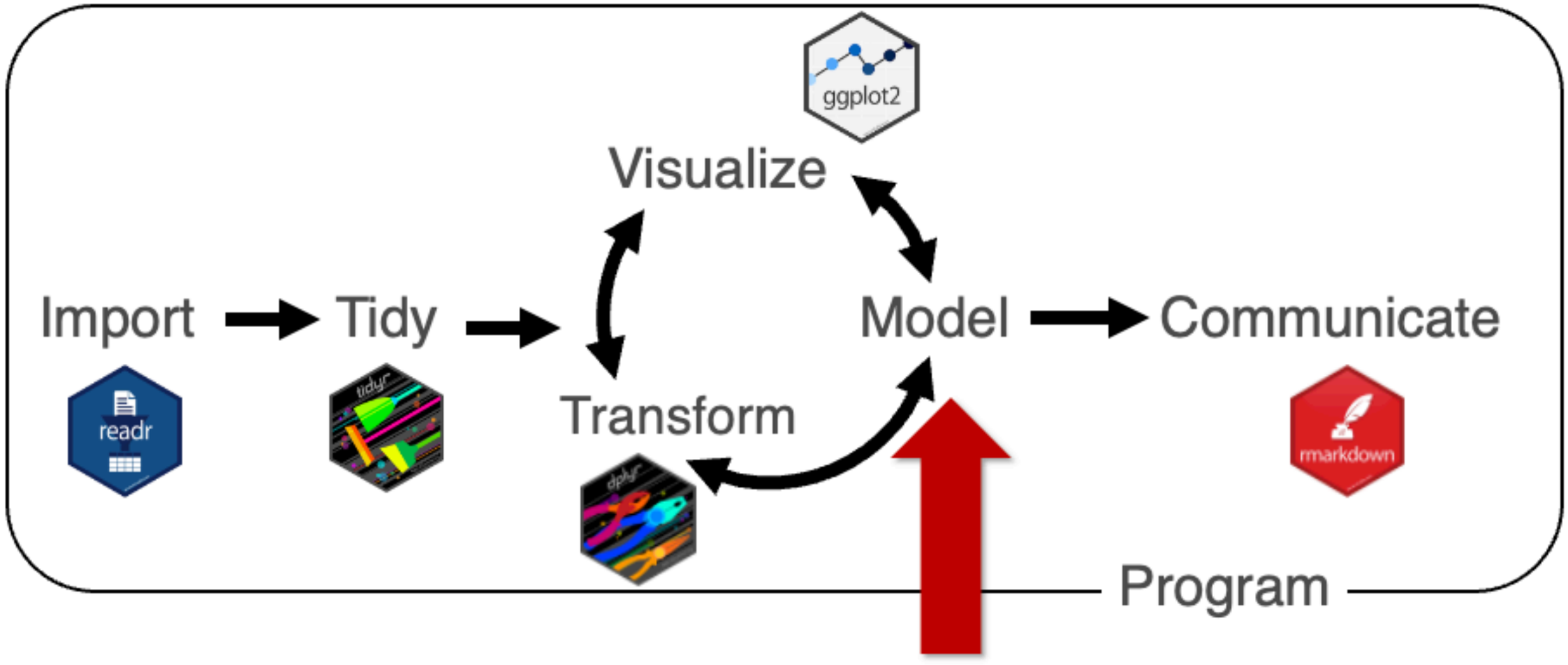
Mêmes données, deux conclusions. R a exécuté les deux tests sans erreur ni message. Une seule réponse est juste, et R ne peut pas savoir laquelle.

1. Décrire des données et **vérifier les conditions** d'un test
2. Formuler une hypothèse, lire une **p-value** et un **intervalle de confiance**
3. **Choisir** le test : t, Wilcoxon, apparié ou non, ANOVA
4. Ajuster et lire une **régression linéaire**
5. Comprendre les **tests multiples** et mener une analyse d'**expression différentielle**

Plan

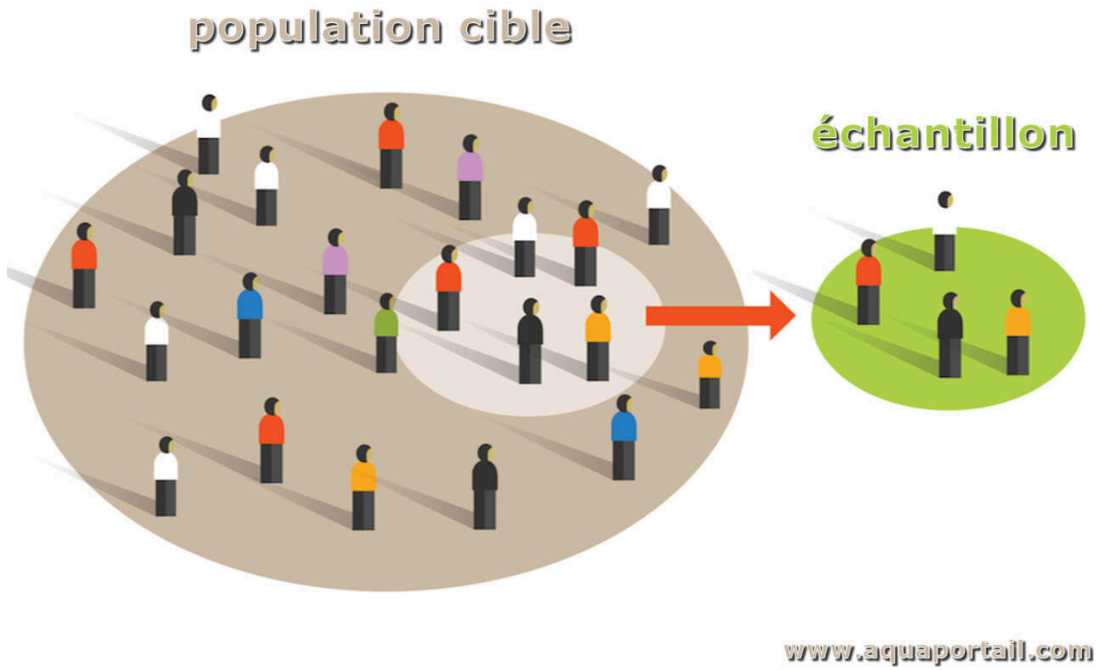
1. Introduction
2. Décrire et vérifier
3. Tester une hypothèse → **TP1**
4. Comparer plusieurs groupes
5. La régression linéaire → **TP2**
6. Tests multiples et expression différentielle → **TP3**

Où en est-on ?



- Modules précédents : **importer, ranger, transformer** (`tidyverse`), **visualiser** (`ggplot2`)
- **Aujourd'hui : modéliser.** Passer de « je vois une différence sur le graphique » à « cette différence n'est probablement pas due au hasard », et dire **de combien**

Échantillon et population



- On mesure un **échantillon** : 8 souris, 4 lignées cellulaires, 344 manchots
- On veut conclure sur la **population** : toutes les souris de cette lignée, tous les manchots de l'archipel
- Un autre échantillon aurait donné d'autres valeurs

> La statistique inférentielle **quantifie l'incertitude** de ce passage de l'échantillon à la population. Elle ne supprime pas cette incertitude.

Les données du jour

Les manchots de l'archipel Palmer (Antarctique), mesurés de 2007 à 2009 : 3 espèces, 344 individus.

`/home/public/stat/penguins.tsv`

```
manchots <- read_tsv("/home/public/stat/per
```

penguins se traduit par « manchots » : les pingouins vivent dans l'hémisphère nord.

```
glimpse(manchots, width = 52)
#> Rows: 344
#> Columns: 8
#> $ species     <chr> "Adelie", "Adelie", "Adelie",
#> $ island      <chr> "Torgersen", "Torgersen", "To
#> $ bill_len    <dbl> 39.1, 39.5, 40.3, NA, 36.7, 3
#> $ bill_dep    <dbl> 18.7, 17.4, 18.0, NA, 19.3, 2
#> $ flipper_len <dbl> 181, 186, 195, NA, 193, 190,
#> $ body_mass   <dbl> 3750, 3800, 3250, NA, 3450, 3
#> $ sex         <chr> "male", "female", "female", NA
#> $ year        <dbl> 2007, 2007, 2007, 2007, 2007,
```

Gorman, Williams et Fraser (2014), *PLoS ONE* ; package `palmerpenguins` (Horst, Hill et Gorman), inclus dans R depuis la version 4.5.

La démarche

1. **Poser la question** avant de regarder les résultats : quels groupes, quelle variable, quel plan d'expérience ?
2. **Décrire et visualiser** : effectifs, moyennes, dispersion, valeurs manquantes, graphiques
3. **Vérifier les conditions** du test envisagé
4. **Tester** : une p-value
5. **Mesurer l'effet** : la différence et son intervalle de confiance
6. **Interpréter** : biologiquement, et avec les limites du plan d'expérience

> R exécute **n'importe quel test sur n'importe quelles données**. Un assistant d'IA écrit `t.test()` en une seconde, mais il ne sait pas que vos patients sont appariés ni comment vos groupes sont codés. Le choix du test, la vérification des conditions et l'interprétation restent votre travail.

2

DÉCRIRE ET VÉRIFIER

Tendance centrale et dispersion

Tendance centrale : où sont les valeurs ?

Mesure	R
Moyenne $\bar{x} = \frac{1}{n} \sum x_i$	`mean(x)`
Médiane (valeur du milieu)	`median(x)`

Dispersion : à quel point varient-elles ?

Mesure	R
Variance, écart-type	`var(x)`, `sd(x)`
Étendue	`range(x)`
Intervalle interquartile (Q3 – Q1)	`IQR(x)`
Quantiles	`quantile(x)`

Sur les manchots, par espèce :

```
manchots |>
  summarise(
    n   = n(),
    moy = mean(body_mass, na.rm = TRUE),
    med = median(body_mass, na.rm = TRUE),
    et  = sd(body_mass, na.rm = TRUE),
    .by = species)
#> # A tibble: 3 × 5
#>   species       n   moy   med    et
#>   <chr>     <int> <dbl> <dbl> <dbl>
#> 1 Adelie      152 3701.  3700  459.
#> 2 Gentoo      124 5076.  5000  504.
#> 3 Chinstrap    68 3733.  3700  384.
```

`na.rm = TRUE` : sinon un seul `NA` rend le résumé `NA` (module 5).

Moyenne ou médiane ?

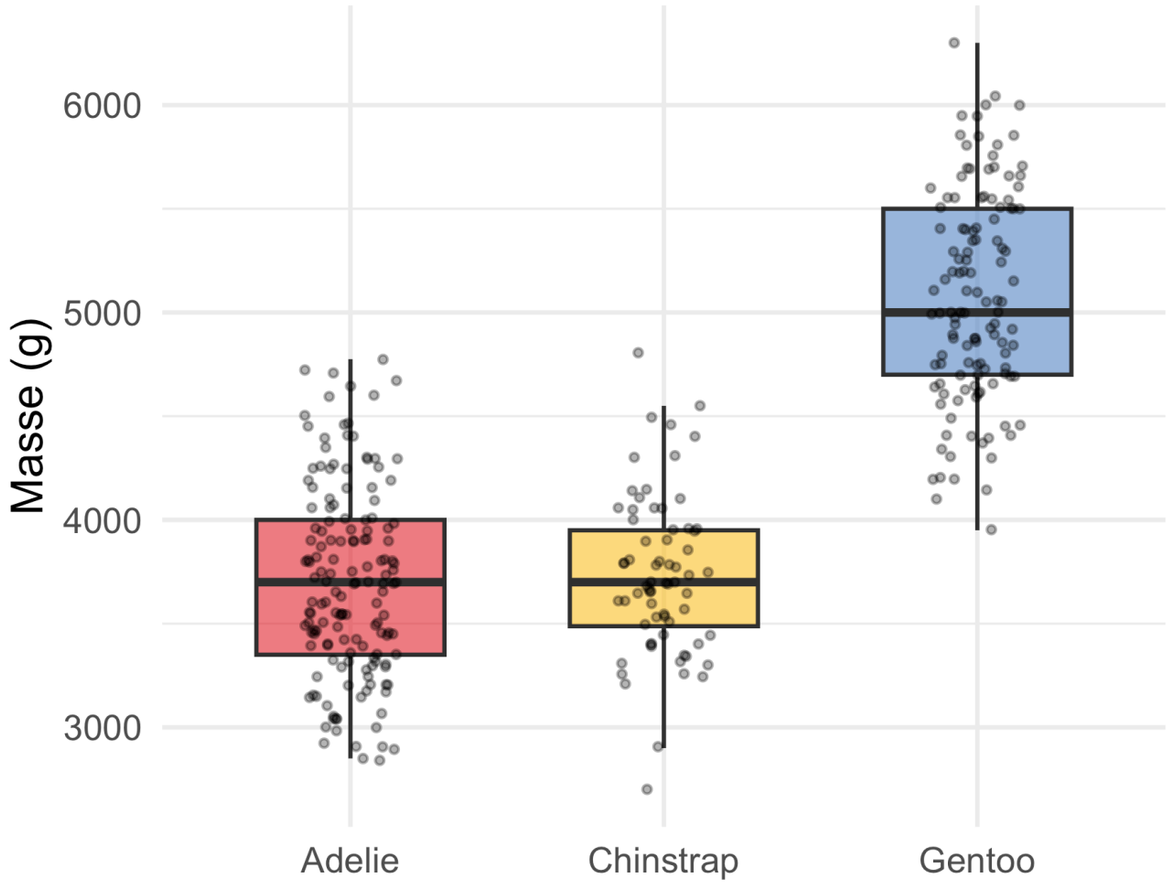
Cinq mesures d'expression, dont une **erreur de saisie** (`51` au lieu de `5.1`) :

```
x <- c(5.1, 4.8, 5.3, 5.0, 51)
mean(x)
#> [1] 14.24
median(x)
#> [1] 5.1
```

- La **moyenne** est tirée par la valeur aberrante, la **médiane** ne bouge presque pas : elle est **robuste**
- Aucun message ne signale le problème : c'est en **décrivant** les données qu'on le voit, avant tout test

> Réflexe : `summary(x)` ou un boxplot avant chaque analyse. Une moyenne très différente de la médiane signale une distribution asymétrique ou une valeur aberrante.

Lire un boxplot

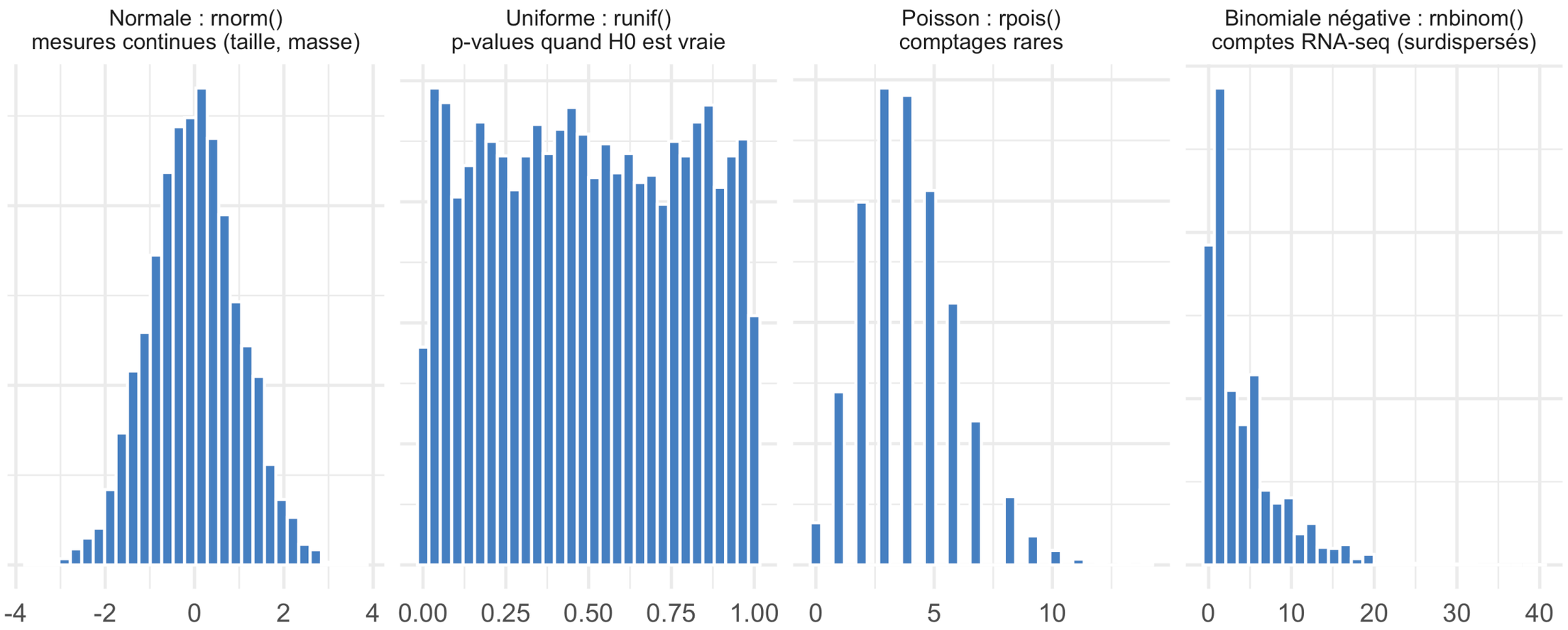


- trait central : la **médiane**
- la boîte : du 1er au 3e quartile (50 % des valeurs)
- les moustaches : jusqu'à 1,5 × IQR au-delà de la boîte
- au-delà : valeurs **atypiques**, à examiner (erreur de saisie ? individu particulier ?)

```
ggplot(manchots, aes(species, body_mass)) +
  geom_boxplot() +
  geom_jitter(width = 0.15)
```

Ajouter les points : avec peu de données, un boxplot seul cache l'effectif.

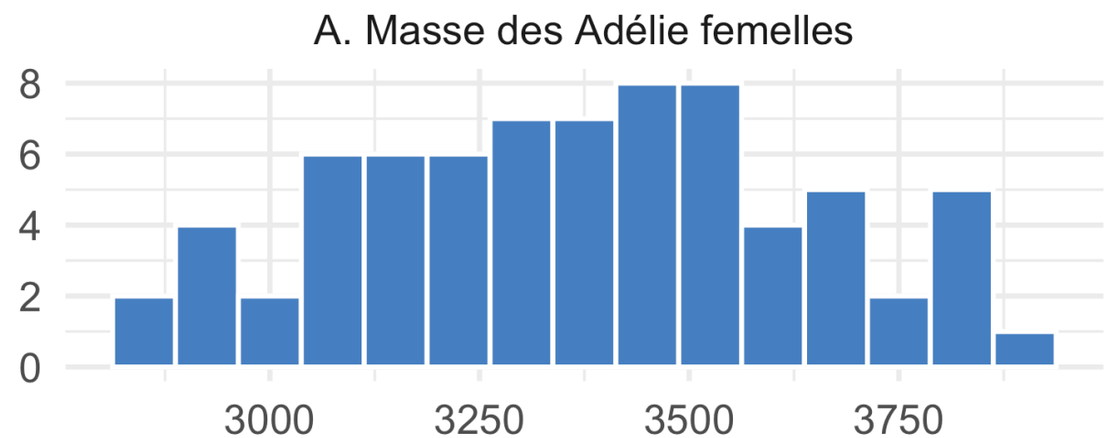
Lois de probabilité usuelles

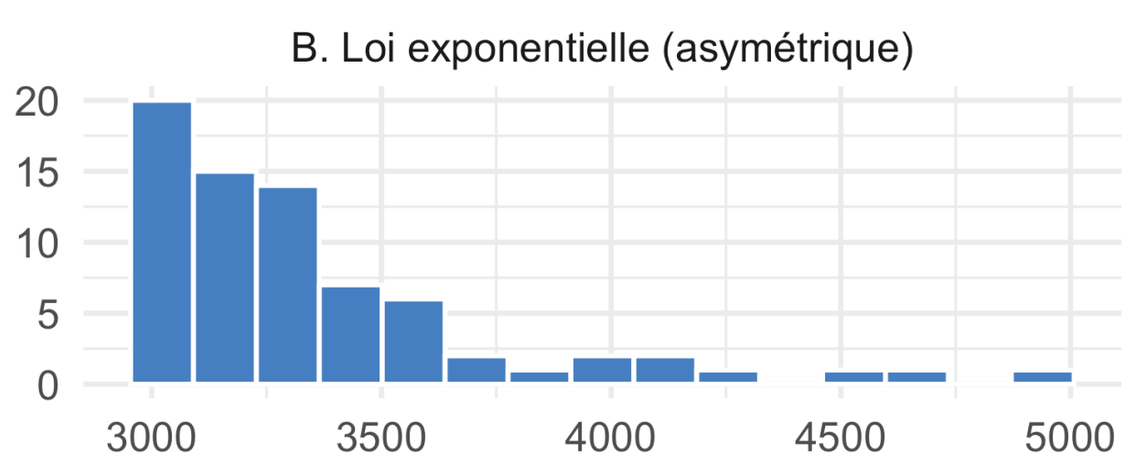


- `rnorm()`, `runif()`, `rpois()`, `rnbinom()` tirent des valeurs au hasard ; `set.seed()` rend le tirage reproductible
- Deux de ces lois reviennent en fin de cours : l'**uniforme** (tests multiples) et la **binomiale négative** (RNA-seq)

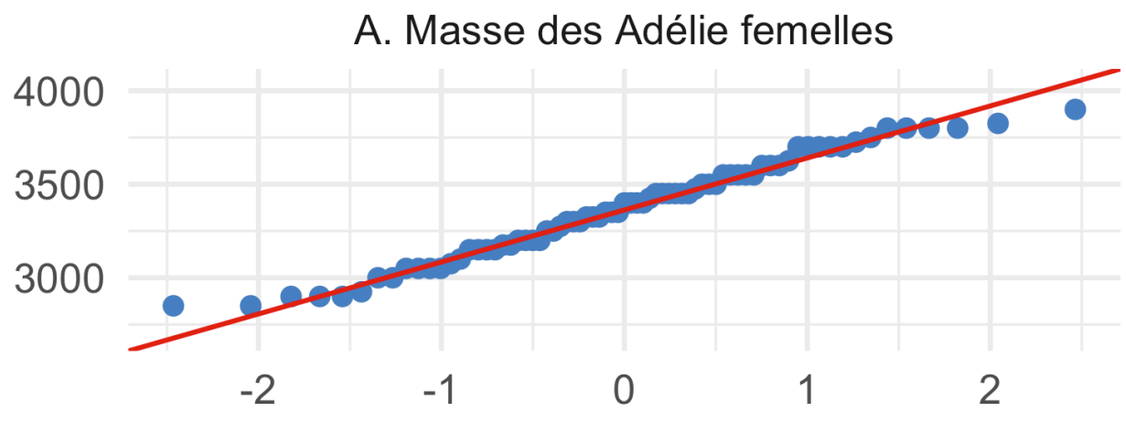
Visualiser la normalité

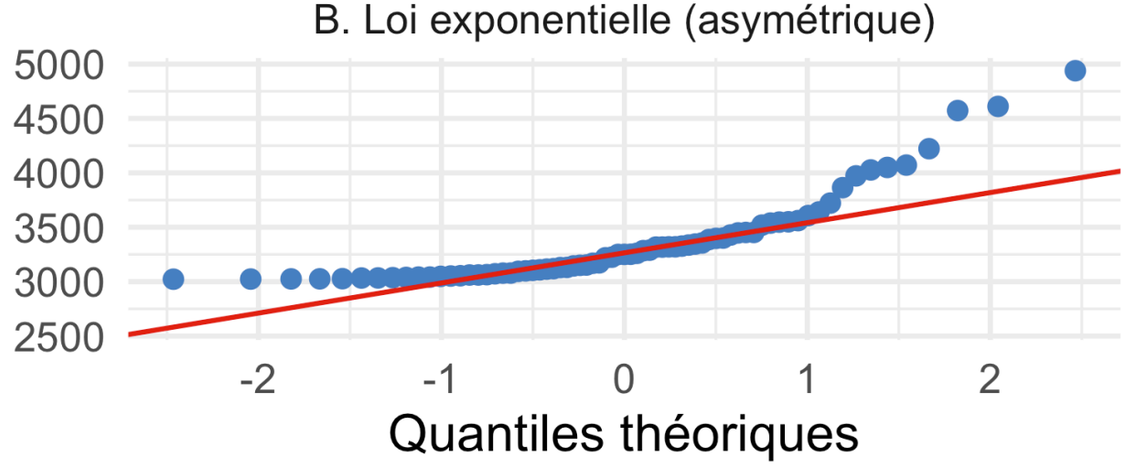
Histogramme





Q-Q plot





Le **Q-Q plot** compare les quantiles observés à ceux d'une loi normale : si les données sont normales, les points suivent la **droite**. A : oui. B : courbure, asymétrie à droite.

```
ggplot(adelie, aes(sample = body_mass)) + stat_qq() + stat_qq_line() + facet_wrap(~ sex)
```

Tester la normalité : Shapiro-Wilk

H0 : les données suivent une loi normale.

```
shapiro.test(adelie$body_mass[adelie$sex == "female"])
#>
#>  Shapiro-Wilk normality test
#>
#> data:  adelie$body_mass[adelie$sex == "female"]
#> W = 0.97684, p-value = 0.1985
```

Par groupe, avec le *tidyverse* :

```
adelie |> summarise(p_shapiro = shapiro.test(body_mass)$p.value, .by = sex)
#> # A tibble: 2 × 2
#>   sex    p_shapiro
#>   <chr>      <dbl>
#> 1 male       0.416
#> 2 female     0.199
```

p > 0.05 : on **ne rejette pas** la normalité, ce qui ne veut pas dire qu'on l'a démontrée.

Ce que Shapiro-Wilk ne dit pas

Peu de données : 1000 échantillons de 8 valeurs tirées d'une loi exponentielle, nettement **non normale**.

```
replicate(1000, shapiro.test(rexp(8))$p.value)
```

Shapiro-Wilk donne p > 0.05 dans **68 %** des cas : il n'a presque pas de puissance.

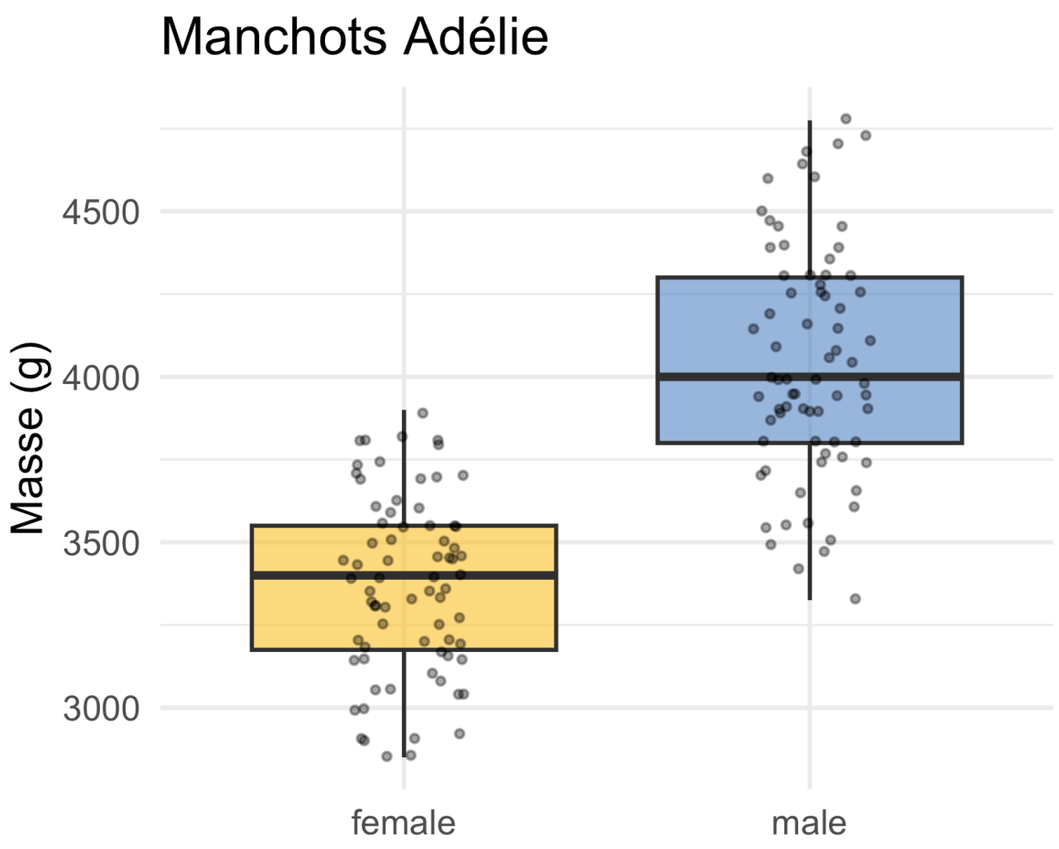
Beaucoup de données : 5000 valeurs d'une loi de Student, presque normale.

```
shapiro.test(rt(5000, df = 10))$p.value
```

p = 3.8e-12 : il détecte un écart **sans importance pratique**.

> Un test de normalité **ne remplace pas le Q-Q plot**. Avec de petits effectifs, il ne détecte presque rien ; avec de grands effectifs, le t-test et l'ANOVA tolèrent bien les écarts à la normalité (théorème central limite). Regardez les données.

Homogénéité des variances



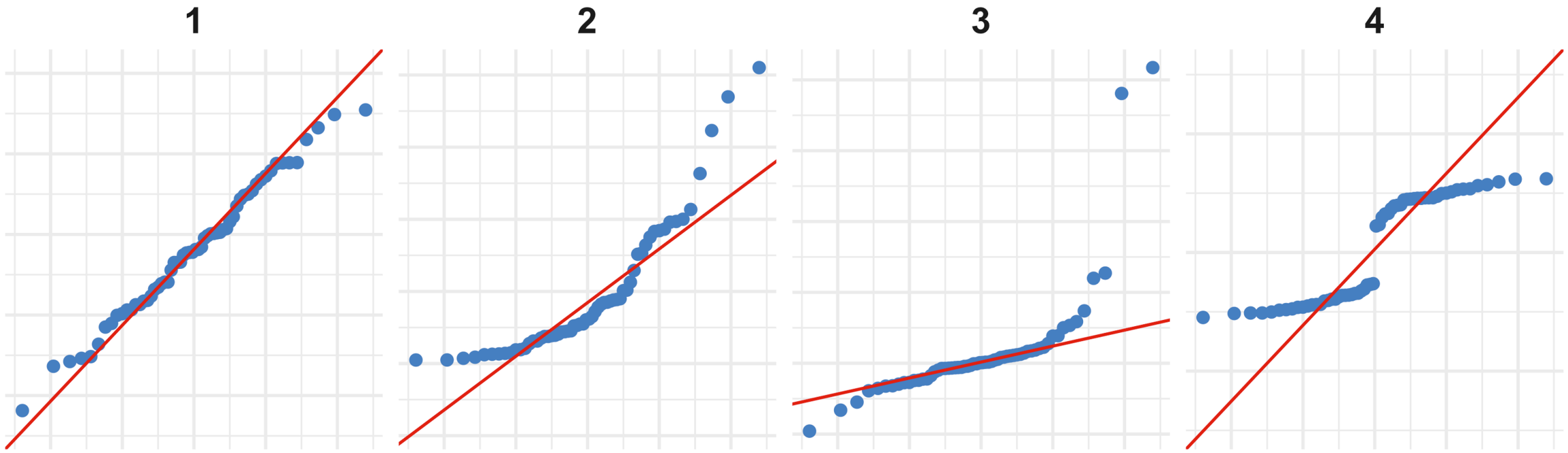
Les boîtes ont-elles des hauteurs comparables ?

Test de Levene (package `car`), H0 : les variances sont égales.

```
car::leveneTest(body_mass ~ factor(sex),
                data = adelie)[1, ]
#> Levene's Test for Homogeneity of Variance (ce
#>       Df F value  Pr(>F)
#> group  1  3.8664 0.05118
```

Ici p ≈ 0.05 : limite. Le t-test de **Welch**, utilisé par défaut dans R, ne suppose pas l'égalité des variances.

✋ À vous : lire des Q-Q plots



Quel jeu de données est compatible avec une loi normale ? Que montrent les trois autres ?

1. **Normal** : les points suivent la droite
2. **Asymétrique à droite** (loi exponentielle) : courbure en « banane »
3. **Queues lourdes** (Student, 2 ddl) : les extrémités s'écartent de part et d'autre, des valeurs extrêmes
4. **Bimodal** (deux groupes mélangés) : deux paliers, un par groupe, reliés par un saut au centre. Un groupe caché dans les données ?

3

TESTER UNE HYPOTHÈSE

La logique d'un test

Question : les mâles et les femelles Adélie ont-ils la **même masse moyenne** ?

- **H0** (hypothèse nulle) : pas de différence, $\mu_F = \mu_M$
- **H1** (hypothèse alternative) : une différence, $\mu_F \neq \mu_M$

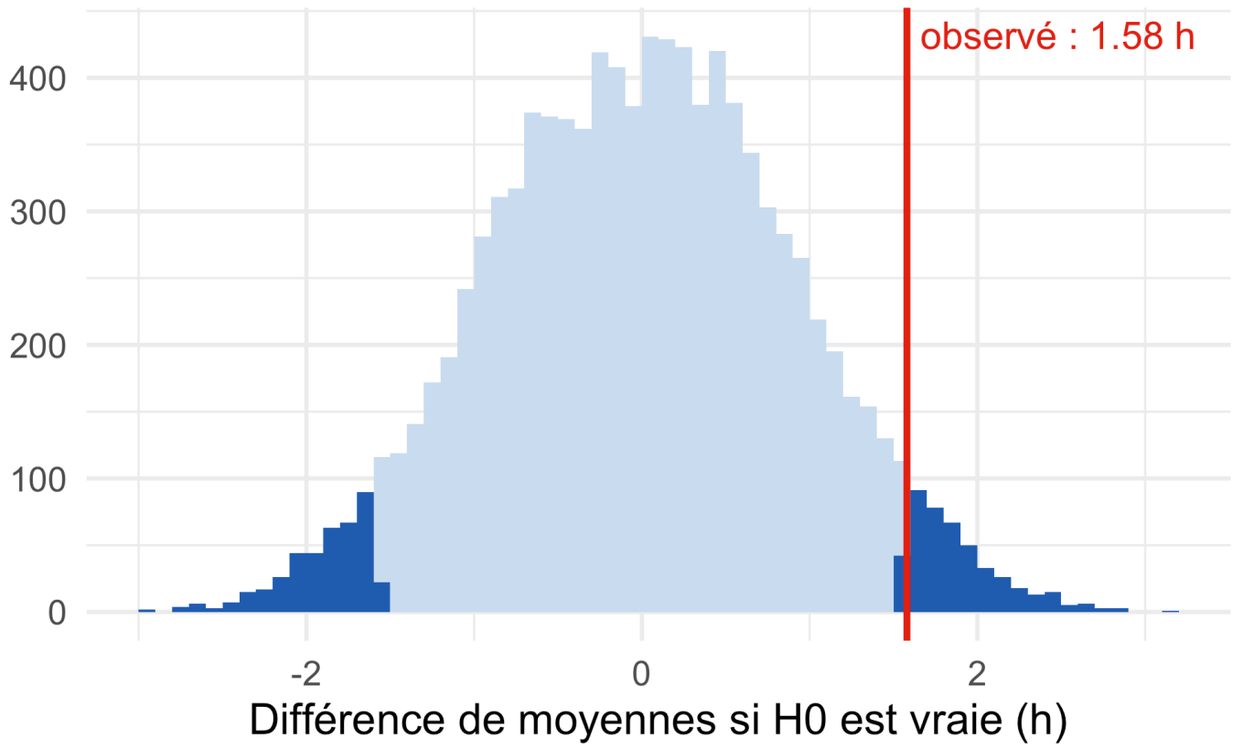
Le raisonnement est **par l'absurde** :

1. Supposons H0 vraie
2. Calculons à quel point la différence observée serait **surprenante** sous H0 : c'est la **p-value**
3. Si elle est très surprenante ($p < \alpha$, souvent 0.05), on **rejette H0**

> On ne « prouve » jamais H1 et on n'« accepte » jamais H0 : on rejette H0, ou on ne la rejette pas.

La p-value, par simulation

Si les deux somnifères avaient **le même effet** (H0), les étiquettes « 1 » et « 2 » seraient interchangeables. Mélangeons-les 10 000 fois :



- en bleu clair : les différences obtenues **par le seul hasard**
- en bleu foncé : celles **au moins aussi extrêmes** que l'observée (1.58 h)
- leur proportion est la **p-value** : 0.0861

`t.test()` donne 0.079 : même idée, calculée avec une loi théorique au lieu de la simulation.

Ce que la p-value n'est pas

La p-value est la probabilité d'observer un écart **au moins aussi grand** que celui observé, **si H0 est vraie**.

Elle n'est **pas** :

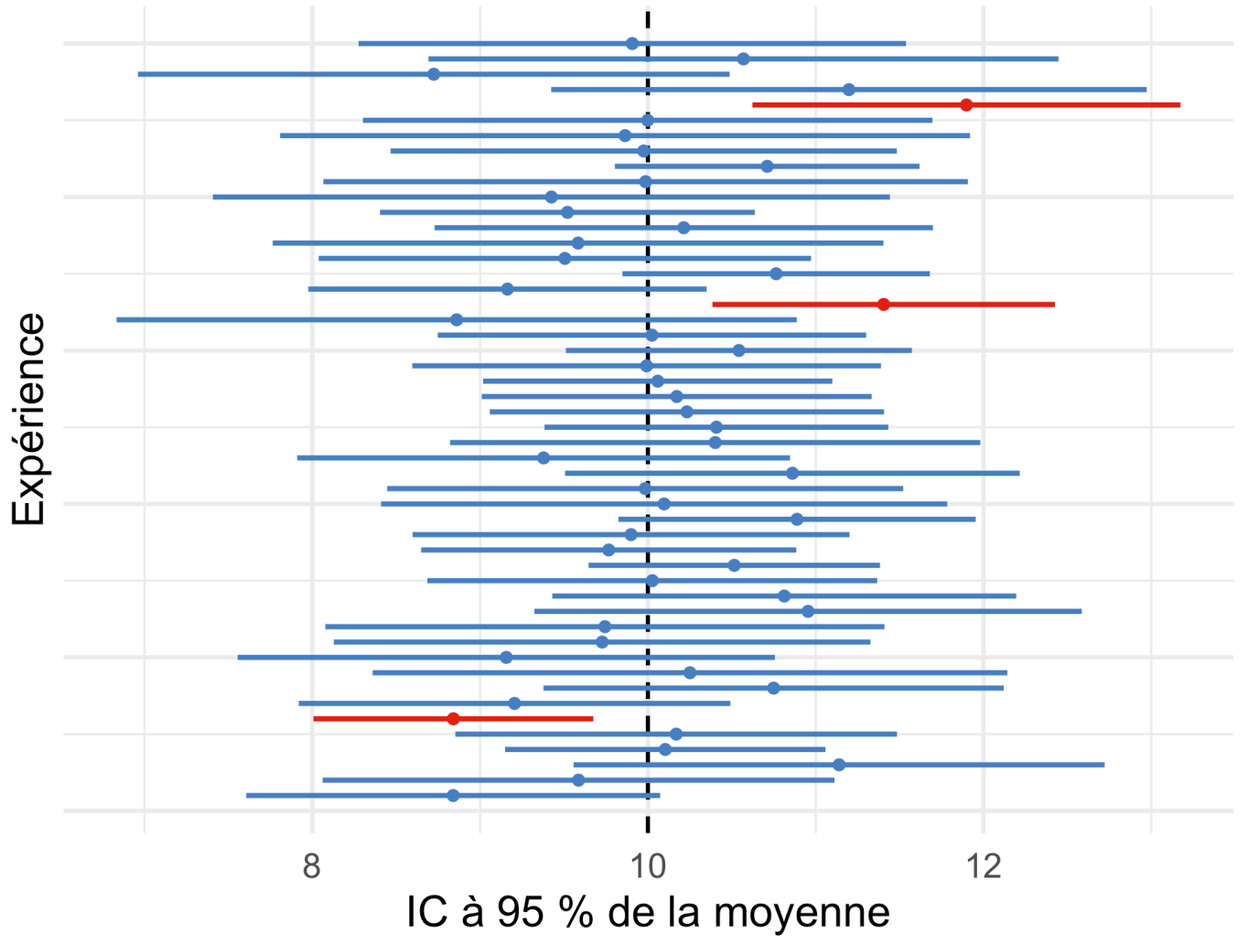
- la probabilité que H0 soit vraie
- la **taille** de l'effet : avec 10 000 individus, une différence minuscule donne p < 0.001
- la probabilité que le résultat se reproduise

Et :

- **p > 0.05 ne veut pas dire « pas de différence »** : peut-être trop peu de données pour la voir. *L'absence de preuve n'est pas la preuve de l'absence.*
- 0.05 est une **convention**, pas une frontière naturelle : p = 0.049 et p = 0.051 disent la même chose

> Rapportez toujours **l'effet** (la différence, avec son intervalle de confiance) en plus de la p-value.

Que veut dire « IC à 95 % » ?



50 expériences simulées de 10 mesures, vraie moyenne **10** (tirets).

- Chaque expérience donne un intervalle différent
- **3 sur 50** manquent la vraie valeur (en rouge) : environ 5 %
- Le « 95 % » porte sur la **méthode** : sur beaucoup d'expériences, 95 % des intervalles contiennent la vraie valeur

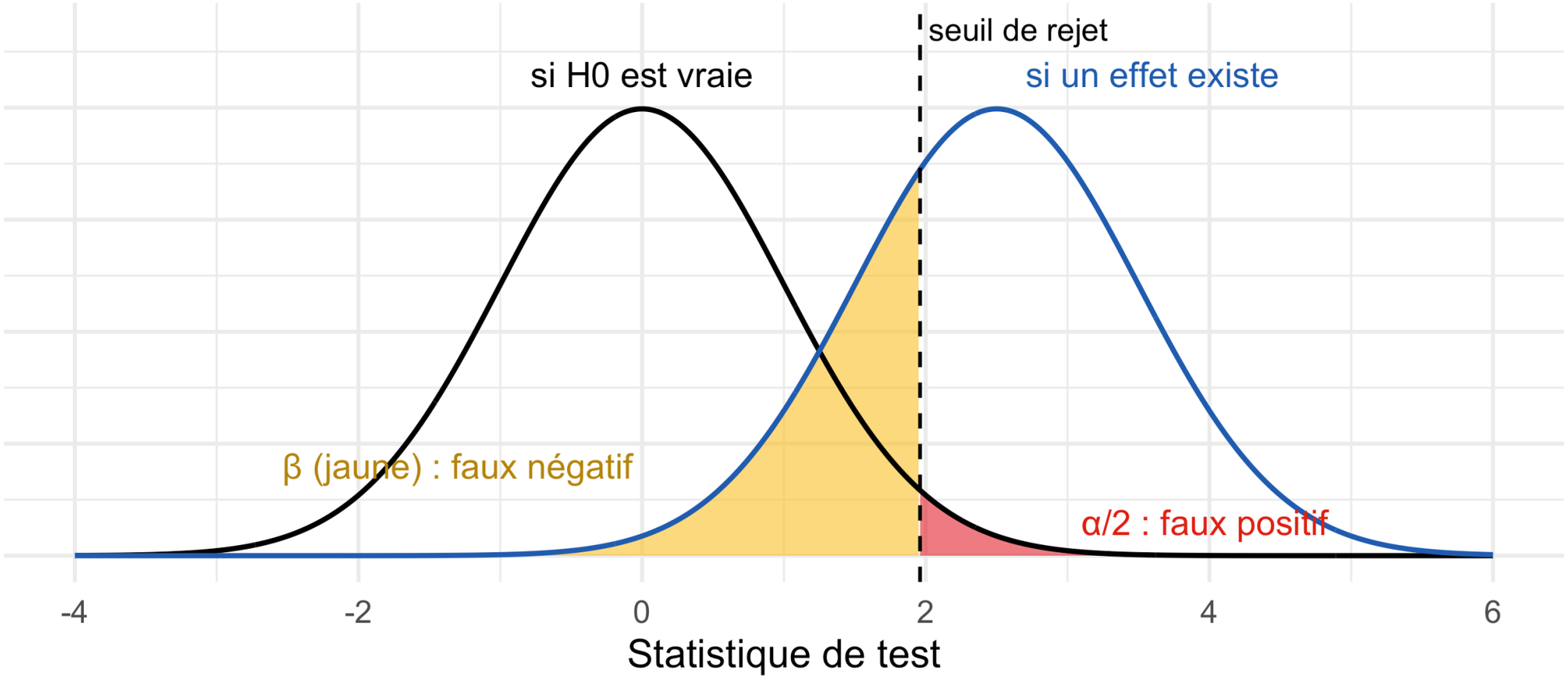
> Un IC étroit dit que l'effet est **précisément** estimé ; un IC large, que les données sont compatibles avec des effets très différents, même si $p < 0.05$.

Erreurs de type I et de type II

	H0 vraie (pas d'effet)	**H0 fausse (un effet)**
On rejette H0	Erreur de **type I** : faux positif (probabilité α)	Bonne décision : **puissance** (1 − β)
On ne rejette pas H0	Bonne décision	Erreur de **type II** : faux négatif (probabilité β)

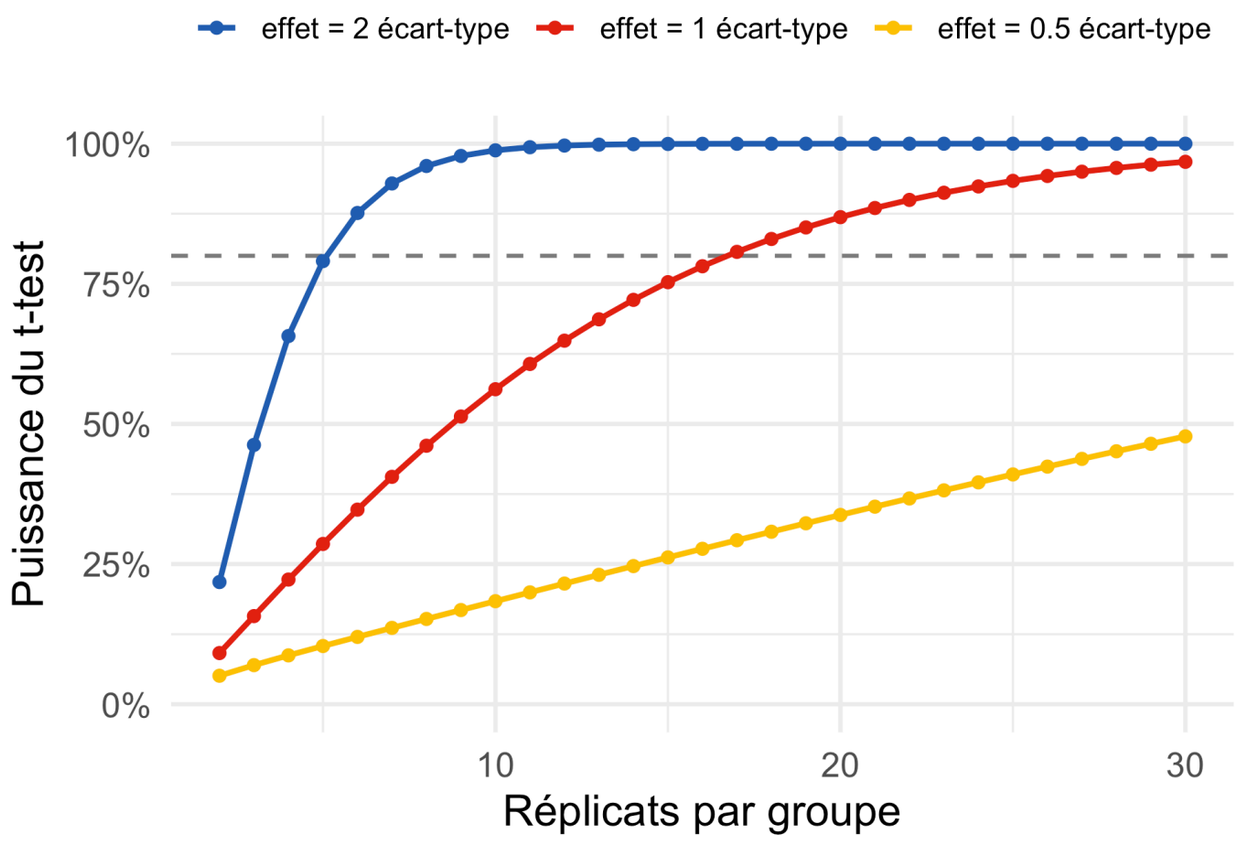
- α est fixé à l'avance (souvent 5 %) : sur 100 tests **sans aucun effet**, environ 5 seront « significatifs »
- La **puissance** augmente avec la taille de l'effet et le nombre de réplicats, et diminue avec la variabilité

α et β en image



- On rejette H0 au-delà du **seuil** : la zone rouge est le risque de faux positif, fixé par α
- La zone jaune est le risque de **manquer** un effet réel (β) : déplacer le seuil vers la droite réduit α mais augmente β
- Plus de réplicats **resserrent** les deux courbes : β diminue sans toucher à α

Combien de réplicats ?



Probabilité de détecter l'effet ($p < 0.05$), selon le nombre de réplicats.

- un effet de 2 écarts-types : **6 réplicats** par groupe pour 80 % de puissance
- un effet d'un écart-type : il en faut **17**
- un effet d'un demi-écart-type : **64** (30 ne donnent que 48 %)

```
power.t.test(delta = 1, sd = 1,
             power = 0.8)
```

Avec 3 réplicats, un résultat non significatif ne dit presque rien. **Le nombre de réplicats se décide avant l'expérience.**

Le t-test

Compare **deux moyennes**.

Situation	Exemple	Commande
Deux groupes **indépendants**	Mâles contre femelles	`t.test(y ~ groupe, data = df)`
Deux mesures **appariées**	Le même patient, avant et après	`t.test(apres, avant, paired = TRUE)`
Un groupe contre une valeur	Moyenne différente de 0 ?	`t.test(x, mu = 0)`

Conditions : observations **indépendantes** ; normalité dans chaque groupe (ou de grands effectifs).

> Par défaut, R fait le **t-test de Welch** (`var.equal = FALSE`), qui ne suppose pas l'égalité des variances. Gardez ce défaut : il perd très peu quand les variances sont égales, et protège quand elles ne le sont pas.

Lire la sortie de `t.test()`

```
t.test(body_mass ~ sex, data = adelie)
#>
#>  Welch Two Sample t-test
#>
#> data:  body_mass by sex
#> t = -13.126, df = 135.69, p-value < 2.2e-16
#> alternative hypothesis: true difference in means between group female and
#> 95 percent confidence interval:
#>  -776.3012 -573.0139
#> sample estimates:
#> mean in group female   mean in group male
#>             3368.836             4043.493
```

- `Welch` : variances non supposées égales
- `t`, `df` : la statistique et ses degrés de liberté
- `p-value` : < 2.2e-16, la plus petite valeur affichée
- `difference ... between group female and group male` : **femelles – mâles**, dans l'ordre des niveaux (alphabétique)
- **intervalle de confiance** : les femelles pèsent de 573 à 776 g de moins
- les deux moyennes

Le test de Wilcoxon (Mann-Whitney)

Alternative **non paramétrique** : travaille sur les **rangs**, pas sur les valeurs.

```
wilcox.test(body_mass ~ sex, data = adelie, exact = FALSE)
#>
#>  Wilcoxon rank sum test with continuity correction
#>
#> data:  body_mass by sex
#> W = 310.5, p-value < 2.2e-16
#> alternative hypothesis: true location shift is not equal to 0
```

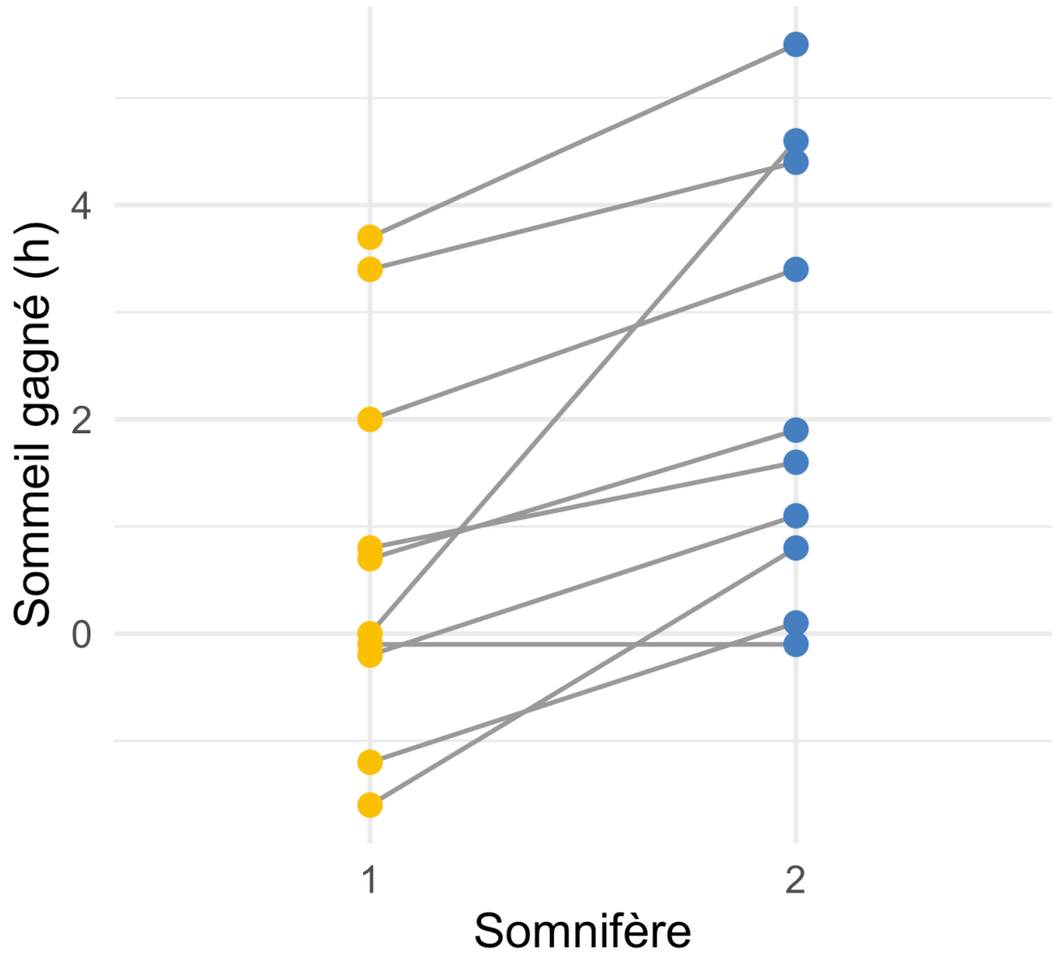
- Ne suppose pas la normalité ; **robuste** aux valeurs aberrantes (`51` au lieu de `5.1` n'est qu'un rang de plus)
- Un peu moins puissant que le t-test quand les données sont normales
- H0 porte sur les **distributions** (une valeur au hasard d'un groupe n'a pas tendance à dépasser celle de l'autre), pas sur les moyennes
- Données appariées : `wilcox.test(apres, avant, paired = TRUE)` (test des rangs signés)

Indépendants ou appariés ?

Les données `sleep` : **les mêmes 10 patients** ont reçu les deux somnifères.

```
sleep
```

```
#>    extra group ID
#> 1    0.7     1  1
#> 2   -1.6     1  2
#> 3   -0.2     1  3
#>     extra group ID
#> 18    1.6     2  8
#> 19    4.6     2  9
#> 20    3.4     2 10
```



Les patients varient beaucoup **entre eux**, mais presque toutes les lignes **montent** : 9 patients sur 10 dorment davantage avec le somnifère 2.

Le même jeu, deux tests

Indépendants (faux ici)

```
t.test(extra ~ group, data = sleep)$p.value
#> [1] 0.07939414
```

Compare deux nuages de points : la variabilité **entre patients** noie l'effet.

Appariés (juste)

```
sommeil <- sleep |>
  pivot_wider(names_from = group, values_from =
              names_prefix = "som_")
t.test(sommeil$som_2, sommeil$som_1,
       paired = TRUE)$p.value
#> [1] 0.00283289
```

Teste les 10 **différences** individuelles : la variabilité entre patients disparaît.

L'appariement n'est pas dans les données, il est dans le plan d'expérience. R ne peut pas le deviner. Deux pièges :

- `t.test(extra ~ group, paired = TRUE)` : erreur dans R ≥ 4.4, `cannot use 'paired' in formula method` (lisez le message)
- apparier deux vecteurs **dans le mauvais ordre** ne produit **aucun message** : c'est pourquoi on passe par `pivot_wider()`, qui aligne les paires par `ID`

Choisir un test

Question	Paramétrique	Non paramétrique
2 groupes **indépendants**	t-test (Welch) : `t.test()`	Mann-Whitney : `wilcox.test()`
2 mesures **appariées**	t-test apparié : `t.test(paired = TRUE)`	rangs signés : `wilcox.test(paired = TRUE)`
3 groupes ou plus	ANOVA : `aov()`	Kruskal-Wallis : `kruskal.test()`
2 variables quantitatives	Pearson : `cor.test()`, régression `lm()`	Spearman : `cor.test(method = "spearman")`
2 variables qualitatives	χ^2 : `chisq.test()`	Fisher : `fisher.test()` (petits effectifs)

Les questions à se poser, dans l'ordre : **quel type de variable** ? **combien de groupes** ? **indépendants ou appariés** ? **conditions vérifiées** ?

✋ À vous : choisir le test

Pour chaque étude, quel test ? Pourquoi ?

1. L'expression d'un gène (qPCR) chez 6 souris *knock-out* et 6 souris sauvages
2. La glycémie de 12 patients, mesurée avant et après un traitement
3. La taille de tumeurs dans 4 groupes de souris recevant 4 doses différentes
4. Un score de douleur de 0 à 10 chez 15 patients, avant et après une chirurgie
5. Le nombre de souris malades ou non, dans un groupe vacciné et un groupe non vacciné

03:00

✋ À vous : choisir le test, correction

1. **Deux groupes indépendants**, petits effectifs : t-test de Welch si le Q-Q plot est acceptable, sinon Mann-Whitney. Avec 6 souris par groupe, la normalité est difficile à vérifier : bien regarder les points
2. **Appariés** (les mêmes patients) : t-test apparié sur les 12 différences, ou rangs signés
3. **Quatre groupes** : ANOVA puis comparaisons deux à deux (Tukey), ou Kruskal-Wallis. La dose doit être un **facteur** (section suivante), ou bien une régression si l'on attend un effet croissant
4. **Apparié et ordinal** (un score de 0 à 10 n'est pas une mesure continue) : test des rangs signés de Wilcoxon
5. **Deux variables qualitatives** (vacciné ou non × malade ou non) : χ^2, ou Fisher si les effectifs sont petits

TP1

4

COMPARER PLUSIEURS GROUPES

Pourquoi pas plusieurs t-tests ?

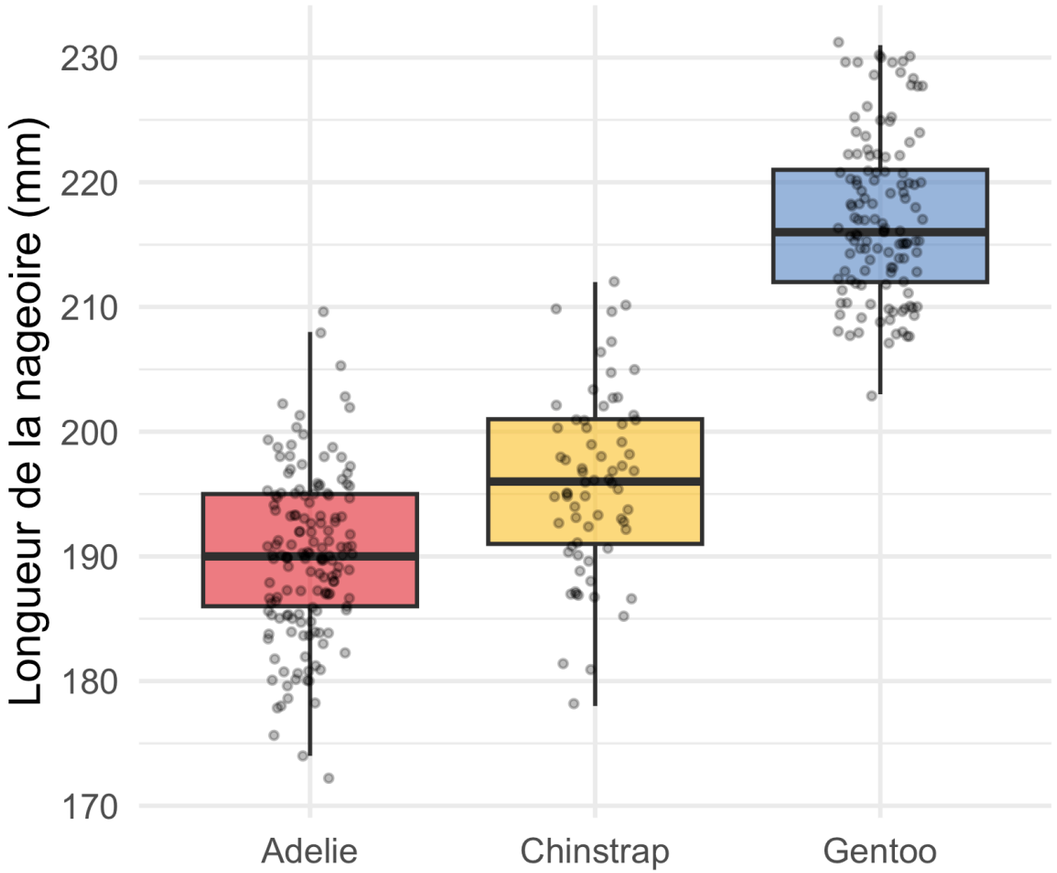
Trois espèces : comparer les nageoires deux à deux fait **3 t-tests**.

Si chaque test a 5 % de risque de faux positif, le risque d'**au moins un** faux positif est :

```
1 - 0.95^3     # 3 groupes : 3 comparaisons
#> [1] 0.142625
1 - 0.95^45    # 10 groupes : 45 comparaisons
#> [1] 0.9005597
```

Plus on fait de tests, plus on est sûr de trouver « quelque chose ». L'**ANOVA** teste toutes les moyennes **en un seul test** ; on ne regarde les paires qu'ensuite, avec une correction. Nous retrouverons ce problème à grande échelle avec 20 000 gènes.

L'ANOVA à un facteur



- **H0** : toutes les moyennes sont égales
- **H1** : au moins une moyenne diffère
- On compare la variabilité **entre** les groupes à la variabilité **à l'intérieur** des groupes : c'est la statistique **F**
- Conditions : indépendance, normalité des **résidus**, variances homogènes (Levene)

L'ANOVA dans R

```
modele_aov <- aov(flipper_len ~ species, data = manchots)
summary(modele_aov)
#>              Df Sum Sq Mean Sq F value Pr(>F)
#> species       2  52473   26237   594.8 <2e-16
#> Residuals   339  14953      44
#> 2 observations deleted due to missingness
```

- `Df` : nombre de groupes − 1 = **2** ; `Residuals` : nombre d'observations − nombre de groupes
- `F value` : variabilité entre groupes / variabilité dans les groupes
- `Pr(>F)` : la p-value. Au moins une espèce a une longueur de nageoire différente
- `2 observations deleted due to missingness` : deux manchots sans mesure, **écartés sans autre avertissement**

Mais lesquelles diffèrent ? L'ANOVA ne le dit pas.

Quels groupes diffèrent ? Le test de Tukey

```
TukeyHSD(modele_aov)
#>   Tukey multiple comparisons of means
#>     95% family-wise confidence level
#>
#> Fit: aov(formula = flipper_len ~ species, data = manchots)
#>
#> $species
#>                         diff       lwr       upr p adj
#> Chinstrap-Adelie  5.869887  3.586583  8.153191     0
#> Gentoo-Adelie    27.233349 25.334376 29.132323     0
#> Gentoo-Chinstrap 21.363462 19.000841 23.726084     0
```

- Une ligne par **paire** : différence (`diff`), intervalle de confiance (`lwr`, `upr`), p-value **ajustée** pour les 3 comparaisons
- Ici, toutes les paires diffèrent. Gentoo a des nageoires plus longues d'environ 27 mm que les Adélie

Version non paramétrique : Kruskal-Wallis

Quand la normalité des résidus est douteuse :

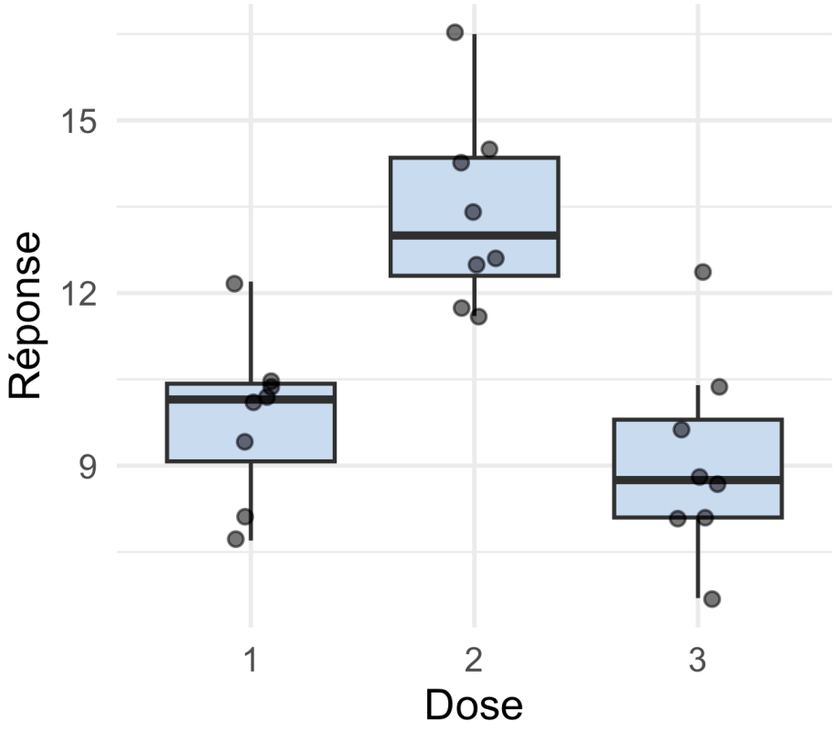
```
kruskal.test(flipper_len ~ species, data = manchots)
#>
#>  Kruskal-Wallis rank sum test
#>
#> data:  flipper_len by species
#> Kruskal-Wallis chi-squared = 244.89, df = 2, p-value < 2.2e-16
```

Puis les paires, avec une **correction** pour les tests multiples :

```
pairwise.wilcox.test(manchots$flipper_len, manchots$species, p.adjust.method = "BH")
#>
#>  Pairwise comparisons using Wilcoxon rank sum test with continuity correction
#>
#> data:  manchots$flipper_len and manchots$species
#>
#>           Adelie Chinstrap
#> Chinstrap 3e-08  -
#> Gentoo    <2e-16 <2e-16
#>
#> P value adjustment method: BH
```

Piège : un groupe codé en nombre

Trois doses codées 1, 2, 3 : une colonne **numérique**.




```r
summary(aov(reponse ~ dose, data = essai))
#>              Df Sum Sq Mean Sq F value Pr(>F)
#> dose          1    2.1   2.103   0.339  0.566
#> Residuals    22  136.5   6.206
```

```r
summary(aov(reponse ~ factor(dose), data = essai))
#>              Df Sum Sq Mean Sq F value   Pr(>F)
#> factor(dose)  2  84.27   42.13   16.27 5.39e-05
#> Residuals    21  54.36    2.59
```

Avec une colonne numérique, `aov()` ajuste une **droite** (1 degré de liberté) et ne voit pas l'effet en cloche. **Aucun message.** Vérification : `Df` doit valoir **nombre de groupes − 1**.

✋ À vous : prédire les degrés de liberté

La colonne `year` des manchots est numérique (2007, 2008 ou 2009), `island` est du texte (3 îles).

Que vaut `Df` (première ligne) dans chaque cas ?

1. `summary(aov(body_mass ~ year, data = manchots))`
2. `summary(aov(body_mass ~ factor(year), data = manchots))`
3. `summary(aov(body_mass ~ island, data = manchots))`

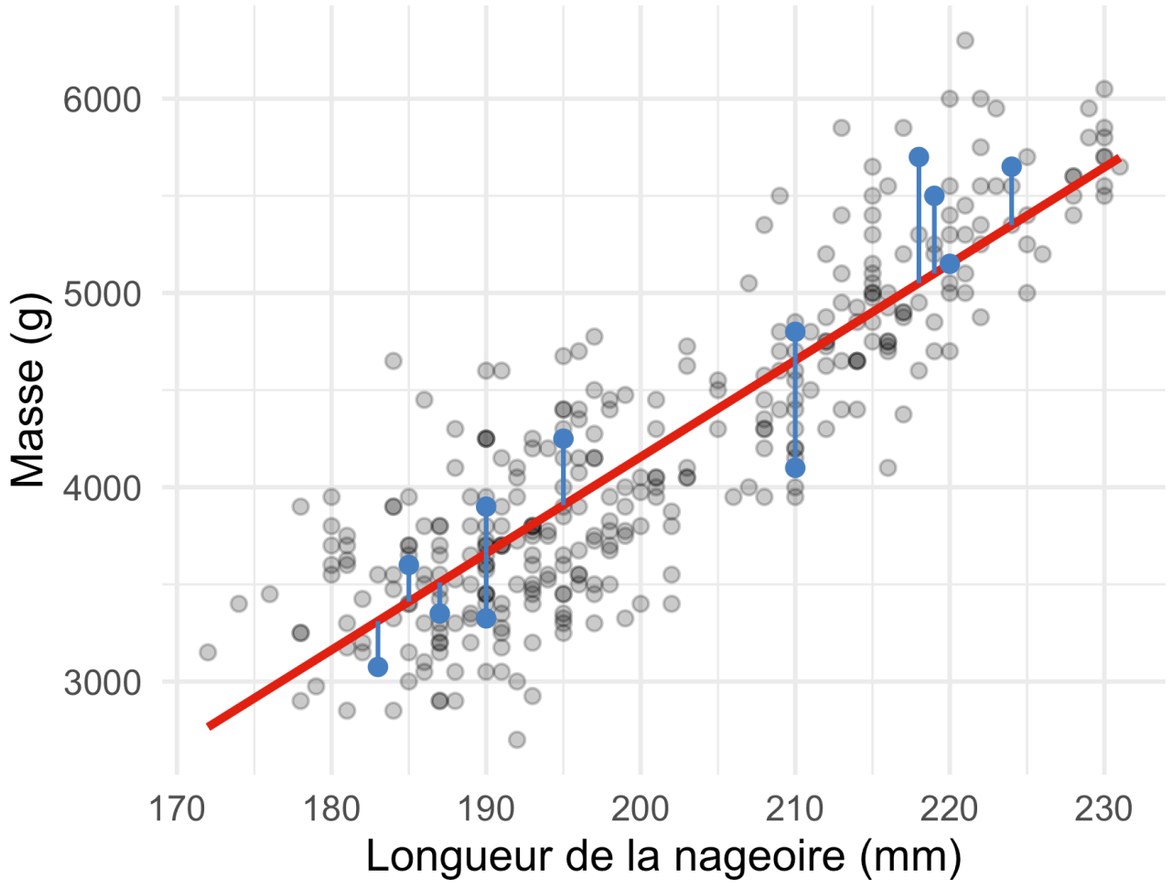
```
summary(aov(body_mass ~ year, data = manchots))[[1]]$Df[1]
#> [1] 1
summary(aov(body_mass ~ factor(year), data = manchots))[[1]]$Df[1]
#> [1] 2
summary(aov(body_mass ~ island, data = manchots))[[1]]$Df[1]
#> [1] 2
```

1 pour l'année numérique (une pente) ; 2 pour l'année en facteur et pour les 3 îles (le texte est converti en facteur).

5

LA RÉGRESSION LINÉAIRE

Le modèle



$$y = \beta_0 + \beta_1 x + \varepsilon$$

- β_0 : l'**ordonnée à l'origine**
- β_1 : la **pente**, la variation de y quand x augmente de 1
- ε : les **résidus** (en bleu), l'écart entre chaque point et la droite

`lm()` choisit la droite qui **minimise la somme des carrés** des résidus.

Régression **multiple** :

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots$$

`lm()` dans R

```
modele <- lm(body_mass ~ flipper_len, data = manchots)
summary(modele)
#>
#> Call:
#> lm(formula = body_mass ~ flipper_len, data = manchots)
#>
#> Residuals:
#>      Min       1Q   Median       3Q      Max
#> -1058.80  -259.27   -26.88   247.33  1288.69
#>
#> Coefficients:
#>              Estimate Std. Error t value Pr(>|t|)
#> (Intercept) -5780.831    305.815  -18.90   <2e-16
#> flipper_len    49.686      1.518   32.72   <2e-16
#>
#> Residual standard error: 394.3 on 340 degrees of freedom
#>   (2 observations deleted due to missingness)
#> Multiple R-squared:  0.759,  Adjusted R-squared:  0.7583
#> F-statistic:  1071 on 1 and 340 DF,  p-value: < 2.2e-16
```

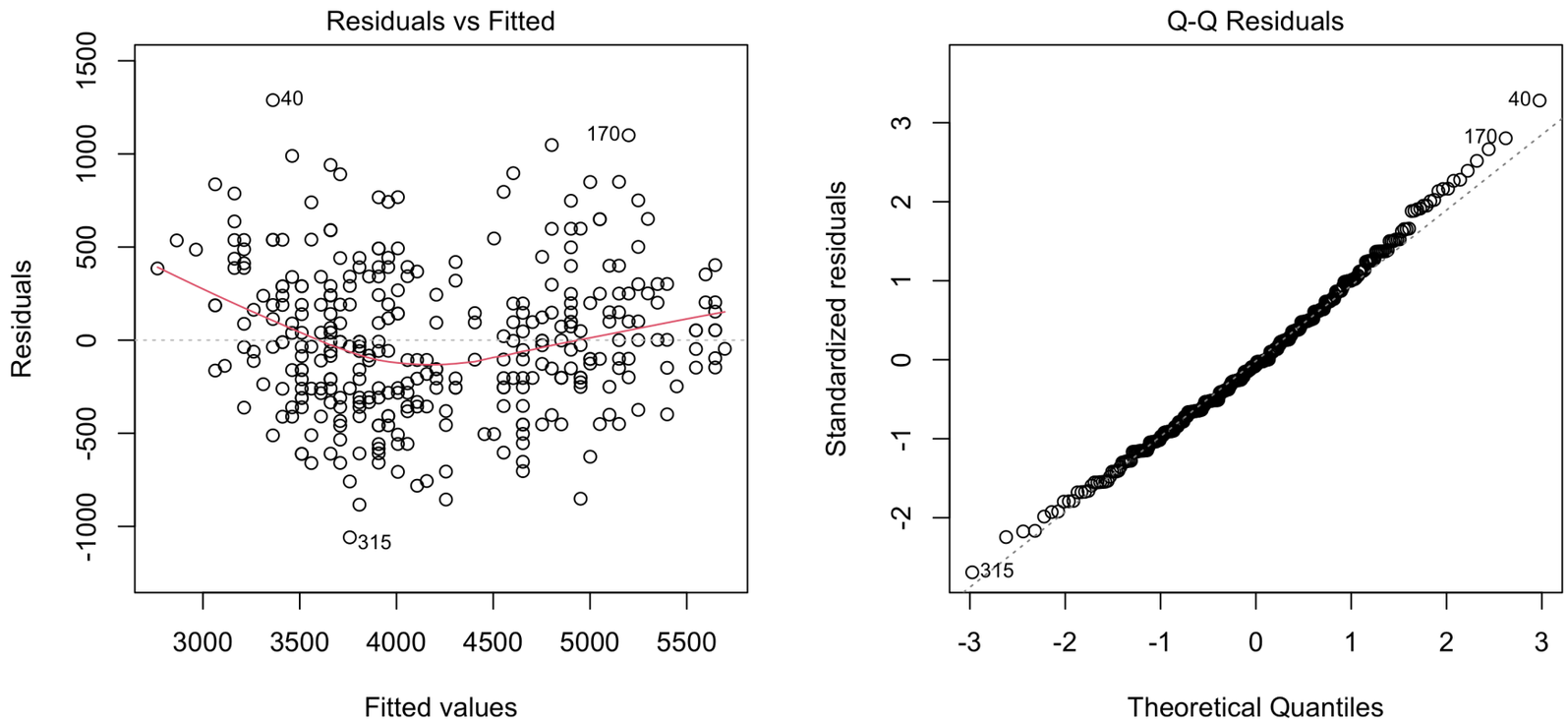
Lire la sortie de `lm()`

- **Pente** (`flipper_len`) : **49.7 g par mm** de nageoire. C'est l'effet : à lire avec son unité
- **Ordonnée à l'origine** : −5781 g pour une nageoire de 0 mm. Ne s'interprète pas : c'est hors des données
- `Pr(>|t|)` de la pente : H0 « pente nulle ». Rejetée
- **R^2** = 0.76 : la longueur de la nageoire explique 76 % de la variance de la masse
- `(2 observations deleted due to missingness)` : à lire, encore

> Une p-value minuscule ne veut pas dire un bon modèle. Avec beaucoup de données, une pente minuscule est « significative » avec un R^2 proche de 0. **Lisez la pente et le R^2, pas seulement les étoiles.**

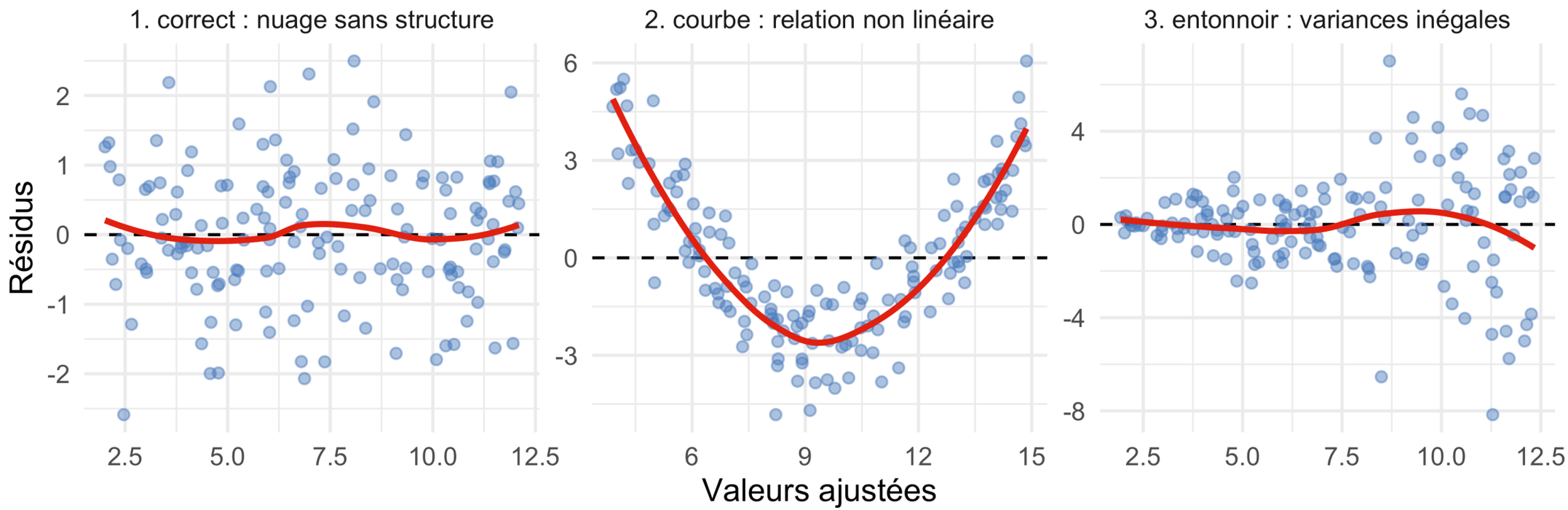
Vérifier les conditions : les résidus

```
par(mfrow = c(1, 2))
plot(modele, which = 1:2)
```



- **Résidus contre valeurs ajustées** : un nuage sans structure. Une courbe signale une relation non linéaire ; un entonnoir, des variances inégales
- **Q-Q plot des résidus** : la normalité porte sur les **résidus**, pas sur y

Lire le graphique des résidus



Données simulées, la même droite `lm(y ~ x)` ajustée trois fois :

1. Ce qu'on espère : les résidus ne dépendent pas de la valeur ajustée
2. Une **courbe** : la relation n'est pas linéaire, la droite se trompe systématiquement aux extrémités et au centre
3. Un **entonnoir** : la dispersion augmente avec la valeur (fréquent pour des concentrations ou des comptes) ; un log aide souvent

Corrélation

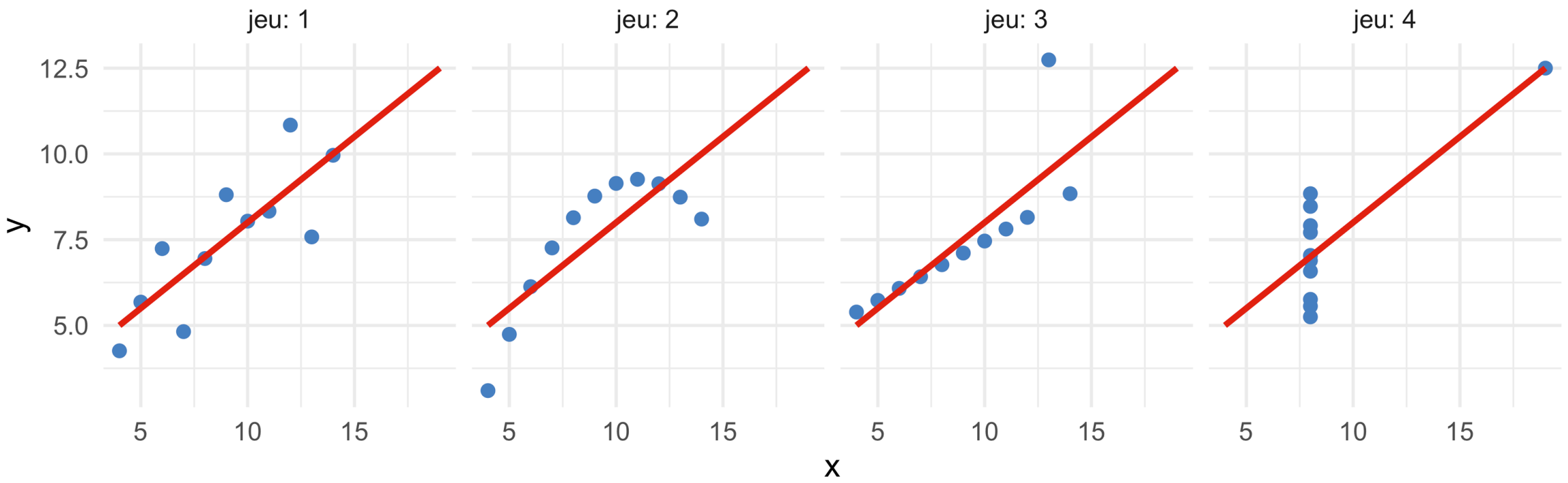
r mesure la force d'une relation **linéaire**, de -1 à 1.

```
cor.test(manchots$flipper_len, manchots$body_mass)
#>
#>  Pearson's product-moment correlation
#>
#> data:  manchots$flipper_len and manchots$body_mass
#> t = 32.722, df = 340, p-value < 2.2e-16
#> alternative hypothesis: true correlation is not equal to 0
#> 95 percent confidence interval:
#>  0.843041 0.894599
#> sample estimates:
#>       cor
#> 0.8712018
```

- **Pearson** (par défaut) : relation linéaire
- **Spearman** (`method = "spearman"`) : sur les rangs, toute relation monotone, robuste
- En régression simple, $R^2 = r^2$: $0.87^2 \approx 0.76$

> Corrélation n'est pas causalité. Les manchots aux longues nageoires sont plus lourds, mais allonger une nageoire ne fera pas grossir un manchot : une **troisième variable** (l'espèce, la taille générale) peut expliquer les deux.

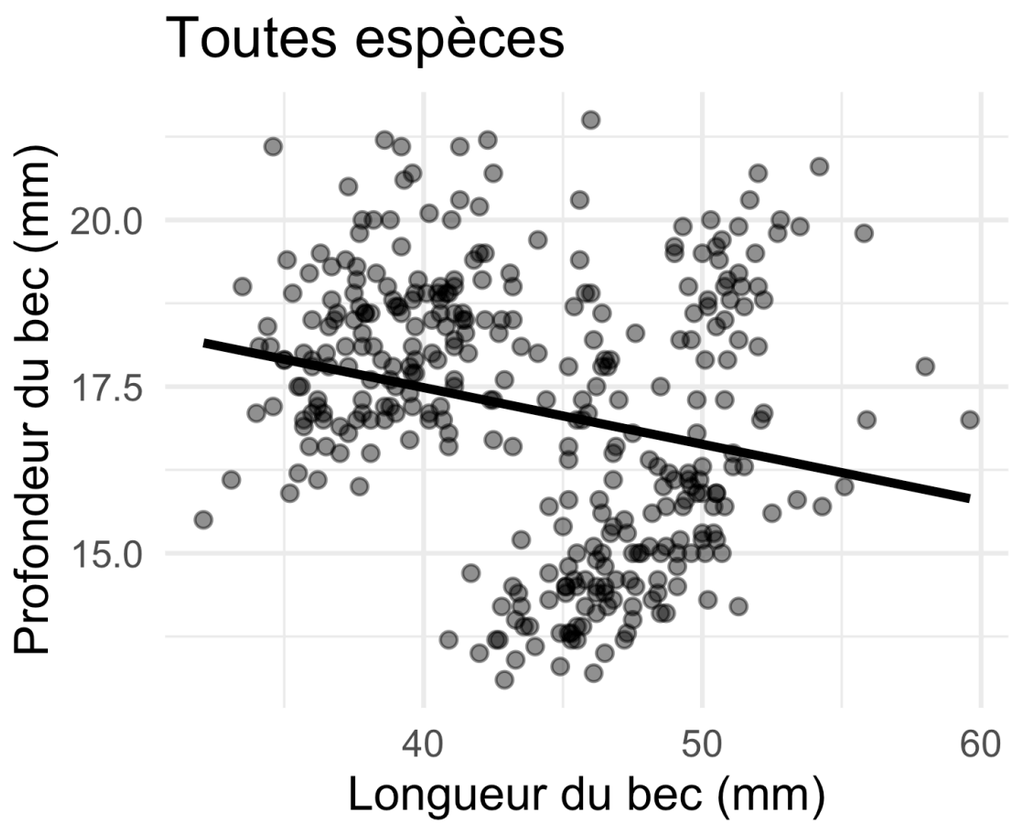
Visualiser avant de résumer : le quartet d'Anscombe

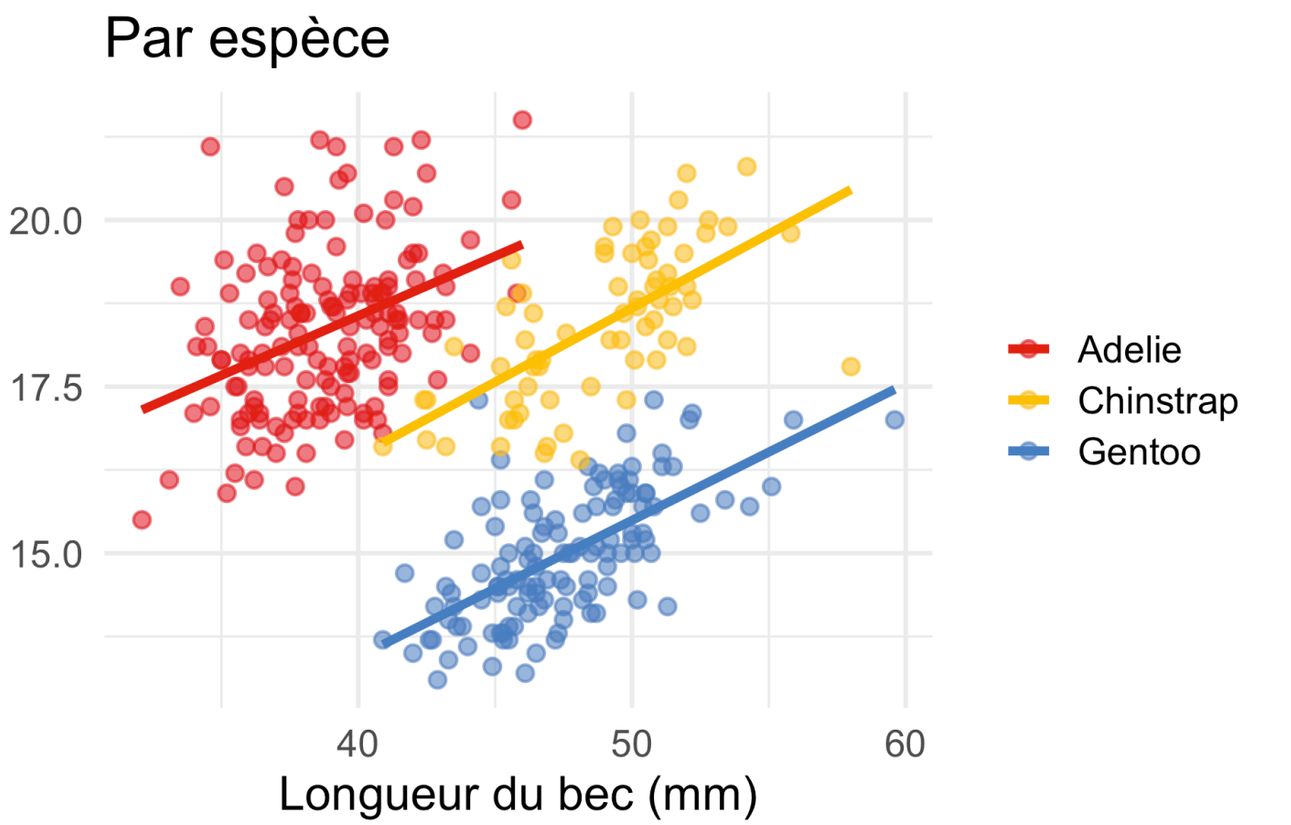


Quatre jeux de données (`anscombe`, F. Anscombe, 1973) avec **les mêmes** moyennes de x et y, les mêmes variances, la même corrélation (0.82) et **la même droite de régression**.

> Seul le jeu 1 se prête à une régression linéaire. Les résumés numériques et les p-values ne voient pas la différence : **le graphique, si**.

Piège : le paradoxe de Simpson






```
round(coef(lm(bill_dep ~ bill_len,
              data = manchots)), 2)
#> (Intercept)    bill_len
#>       20.89       -0.09
```

Pente **négative** : un bec long serait moins profond ?

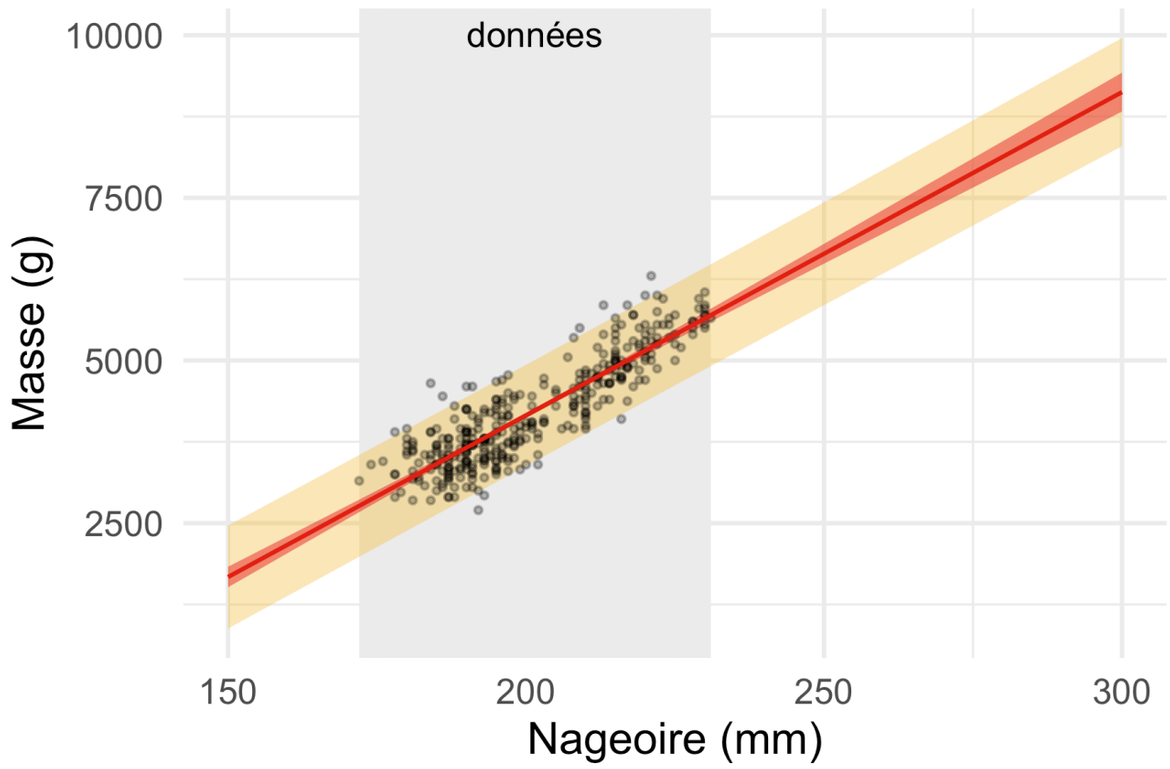
```
round(coef(lm(bill_dep ~ bill_len + species,
              data = manchots))[1:2], 2)
#> (Intercept)    bill_len
#>       10.59        0.20
```

Pente **positive** dans chaque espèce : l'espèce, **oubliée**, inversait la conclusion.

Ne pas extrapoler

```
range(manchots$flipper_len, na.rm = TRUE)
#> [1] 172 231
predict(modele, newdata =
  tibble(flipper_len = c(200, 300)))
#>        1        2
#> 4156.282 9124.839
```

- 200 mm : dans les données, la prédiction a un sens (≈ 4.2 kg)
- 300 mm : **hors des données**, R prédit un manchot de 9 kg, sans aucun avertissement



rouge : IC de la droite ; jaune : intervalle de **prédiction** d'un manchot

Le modèle n'est valable que **dans l'étendue des données** qui ont servi à l'ajuster. R ne le sait pas, vous oui.

✋ À vous : lire une régression multiple

```
modele_bec <- lm(bill_dep ~ bill_len + species, data = manchots)
round(coef(summary(modele_bec)), 3)
#>                  Estimate Std. Error t value Pr(>|t|)
#> (Intercept)        10.592      0.683  15.508        0
#> bill_len            0.200      0.017  11.427        0
#> speciesChinstrap   -1.933      0.224  -8.624        0
#> speciesGentoo      -5.106      0.191 -26.674        0
```

1. Comment interpréter le coefficient de `bill_len` ?
2. Que signifie le coefficient `speciesGentoo` ? Par rapport à quelle espèce ?
3. Pourquoi n'y a-t-il pas de ligne `speciesAdelie` ?

1. **À espèce égale**, 1 mm de longueur de bec en plus correspond à 0.20 mm de profondeur en plus
2. À longueur de bec égale, le bec des Gentoo est **5.1 mm moins profond** que celui des **Adélie**
3. Adélie est le **niveau de référence** (premier dans l'ordre alphabétique) : il est dans l'ordonnée à l'origine, et les autres espèces s'y comparent

TP2

6

TESTS MULTIPLES ET EXPRESSION DIFFÉRENTIELLE

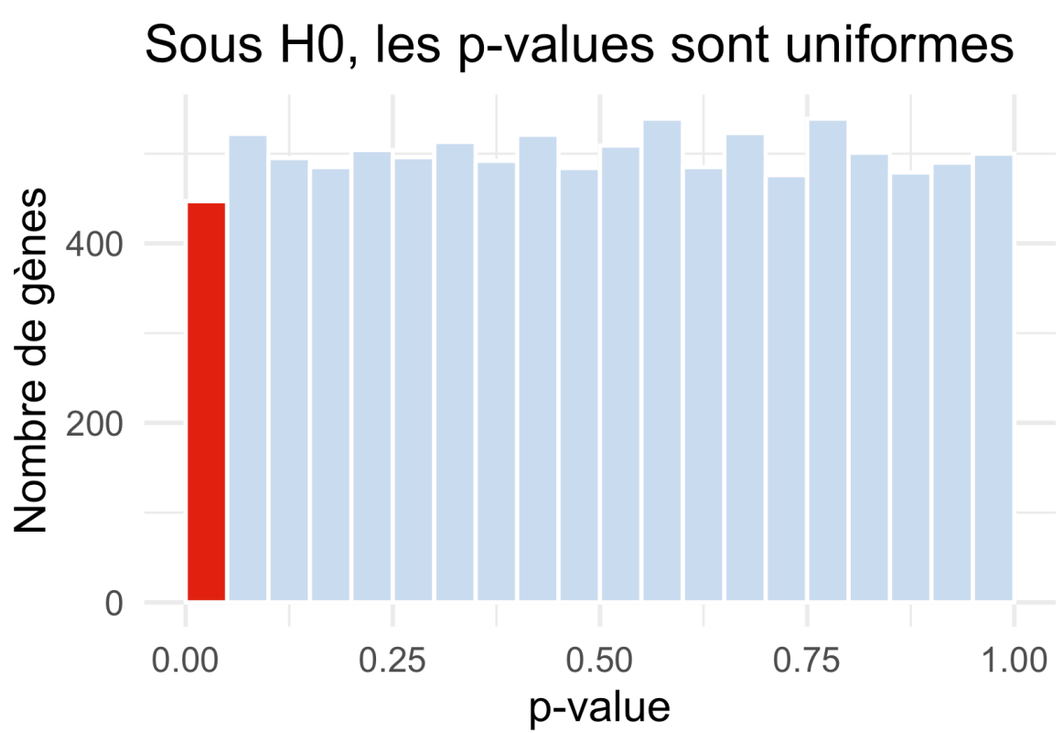
10 000 gènes sans aucun effet

Simulons 10 000 gènes **identiques** dans deux groupes de 5 échantillons, et faisons un t-test par gène :

```
expr <- matrix(rnorm(10000 * 10), ncol = 10)
p <- apply(expr, 1, \(x) t.test(x[1:5], x[6:10])
sum(p < 0.05)
```

```
#> [1] 447
```

447 gènes « significatifs », et **tous** sont des faux positifs.



α = 5 % **par test** : sur 20 000 gènes humains, environ **1000 faux positifs**, même si aucun gène ne réagit.

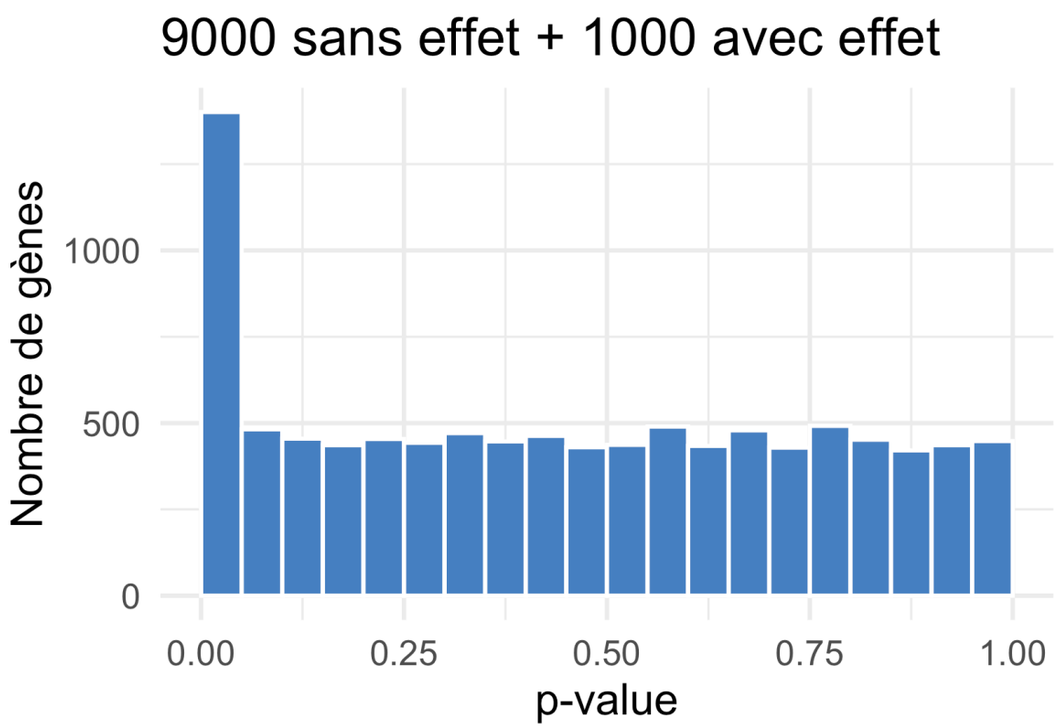
Corriger pour les tests multiples

Méthode	Contrôle	Idée	R
Bonferroni	FWER : probabilité d'**au moins un** faux positif	multiplier chaque p-value par le nombre de tests	`p.adjust(p, "bonferroni")`
Benjamini-Hochberg	**FDR** : **proportion** de faux positifs parmi les gènes retenus	ajustement selon le rang de la p-value	`p.adjust(p, "BH")`

```
sum(p.adjust(p_nul, method = "bonferroni") < 0.05)
#> [1] 0
sum(p.adjust(p_nul, method = "BH") < 0.05)
#> [1] 0
```

- **Bonferroni** est très strict : adapté à quelques tests, trop sévère pour 20 000 gènes
- **FDR (BH)** : le standard en génomique. « FDR < 5 % » : parmi les gènes retenus, on accepte qu'environ 5 % soient de faux positifs
- La p-value ajustée s'appelle `padj` dans DESeq2, ou *q-value*

Avec de vrais effets

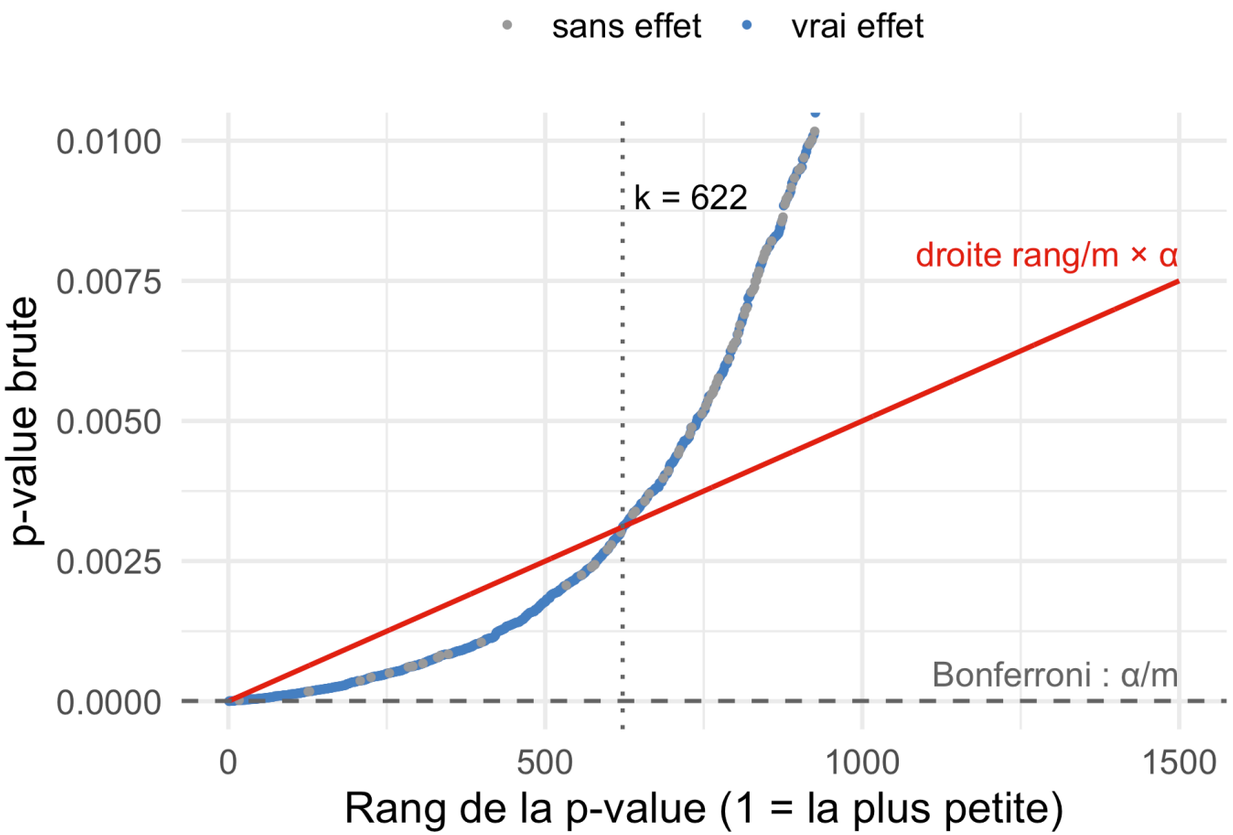


Les vrais effets forment un **pic près de 0**, au-dessus du fond uniforme.

Seuil 0.05 sur	Retenus	Vrais	Faux positifs
p brute	1401	979	422
`padj` (BH)	622	602	20
Bonferroni	6	6	0

> Regarder l'**histogramme des p-values** est le premier contrôle d'une analyse à grande échelle : plat = aucun signal ; pic près de 0 = du signal.

La procédure de Benjamini-Hochberg



Sur la même simulation (m = 10 000 tests, α = 0.05) :

1. Trier les p-values de la plus petite à la plus grande
2. Comparer chacune à la droite **rang / m × α**
3. Retenir toutes les p-values jusqu'au **dernier** point sous la droite : ici **k = 622** gènes

Bonferroni compare **toutes** les p-values au même seuil α/m (tirets), minuscule : il n'en retient que 6.

> `p.adjust(p, "BH")` fait ce calcul pour vous : `padj < 0.05` revient exactement à cette règle.

Les données RNA-seq : `airway`

Cellules musculaires lisses des voies respiratoires de **4 donneurs** (lignées), chacune **traitée ou non** à la dexaméthasone, un corticoïde utilisé contre l'asthme.

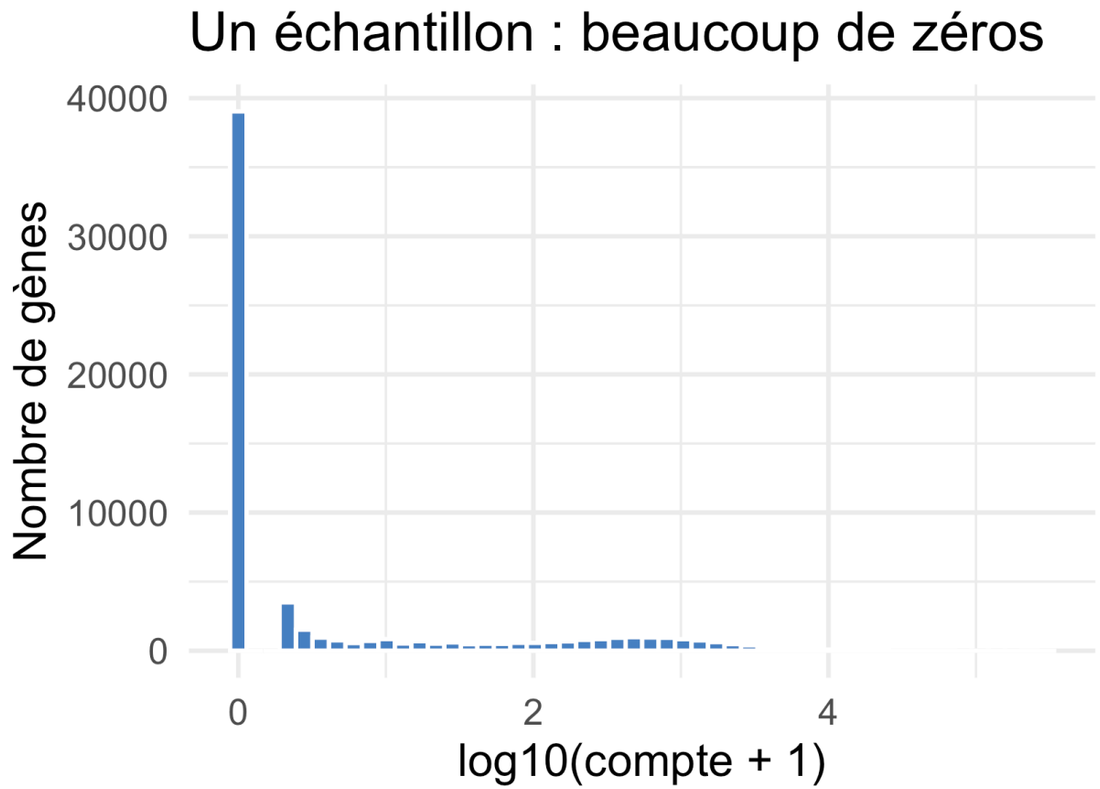
```
comptes <- read_tsv("/home/public/stat/airway_counts.tsv") |>
  column_to_rownames("gene") |>
  as.matrix()
infos <- read_tsv("/home/public/stat/airway_sample_info.tsv") |>
  select(Run, cell, dex)
```

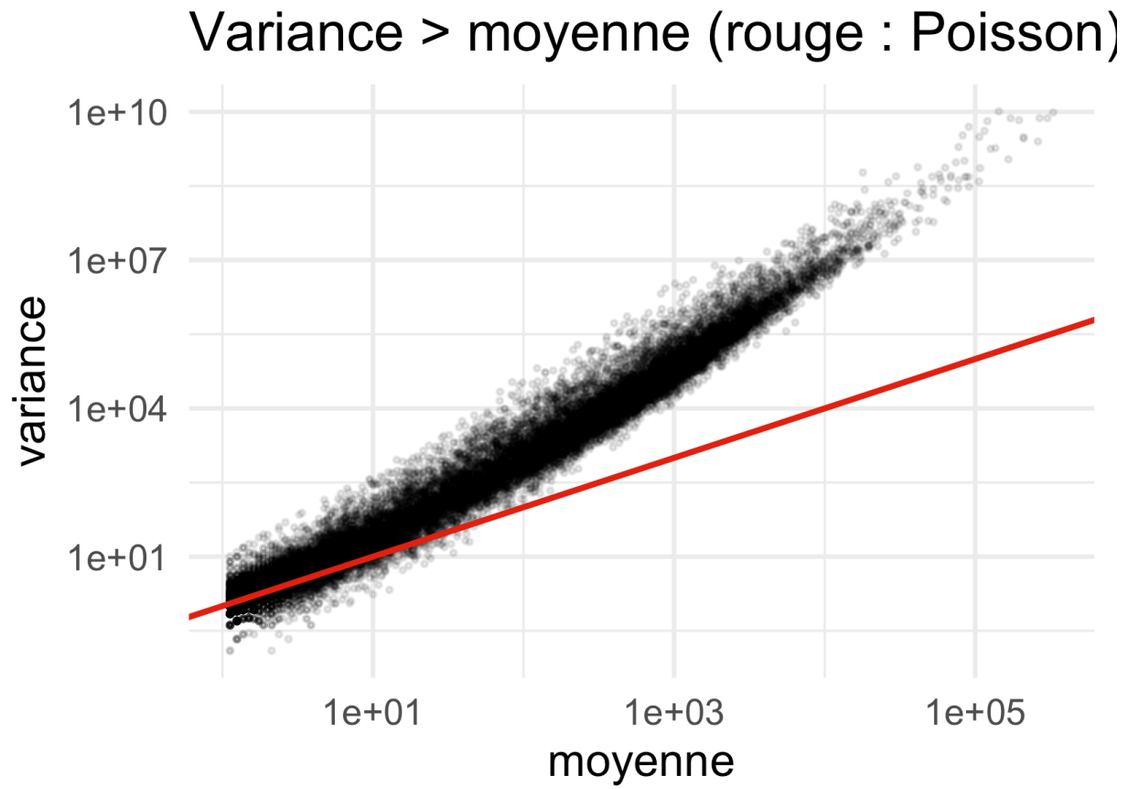
```
dim(comptes)
#> [1] 63677     8
comptes[1:3, 1:4]
#>                 SRR1039508 SRR1039509 SRR1039512 SRR1039513
#> ENSG00000000003        679        448        873        408
#> ENSG00000000005          0          0          0          0
#> ENSG00000000419        467        515        621        365
```

```
infos
#> # A tibble: 8 × 3
#>   Run        cell    dex
#>   <chr>      <chr>   <chr>
#> 1 SRR1039508 N61311  untrt
#> 2 SRR1039509 N61311  trt
#> 3 SRR1039512 N052611 untrt
#> 4 SRR1039513 N052611 trt
#> 5 SRR1039516 N080611 untrt
#> 6 SRR1039517 N080611 trt
#> 7 SRR1039520 N061011 untrt
#> 8 SRR1039521 N061011 trt
```

Himes *et al.* (2014), *PLoS ONE* ; package Bioconductor `airway`.

Des comptes, pas des mesures continues





- Des **entiers** ≥ 0, très asymétriques, beaucoup de zéros : pas une loi normale
- La variance **dépasse** la moyenne (surdispersion) : on utilise une loi **binomiale négative**
- La **profondeur de séquençage** diffère d'un échantillon à l'autre (`colSums(comptes)`) : il faut **normaliser**
- 4 réplicats par condition : trop peu pour estimer la variance de chaque gène seul. DESeq2 **partage l'information** entre gènes

Le plan d'expérience : `design`

Run	cell	dex
SRR1039508	N61311	untrt
SRR1039509	N61311	trt
SRR1039512	N052611	untrt
SRR1039513	N052611	trt
SRR1039516	N080611	untrt
SRR1039517	N080611	trt
SRR1039520	N061011	untrt
SRR1039521	N061011	trt

- 4 lignées × 2 conditions : **chaque lignée est son propre témoin**, comme les patients de `sleep`
- `design = ~ cell + dex` : tenir compte de la lignée, puis tester le traitement (la **dernière** variable)
- `design = ~ dex` ignorerait l'appariement : la variabilité entre donneurs noierait une partie de l'effet

Le niveau de référence

La comparaison se fait **par rapport au premier niveau** du facteur. Par défaut, l'ordre est **alphabétique** :

```
levels(factor(infos$dex))
#> [1] "trt"   "untrt"
```

`"trt"` vient avant `"untrt"` : sans précaution, le **traité** serait la référence, et tous les log2 *fold changes* seraient **inversés**.

```
infos <- infos |>
  mutate(dex = factor(dex, levels = c("untrt", "trt")),   # le non-traité est la référence
         cell = factor(cell))
levels(infos$dex)
#> [1] "untrt" "trt"
```

Même règle que `speciesGentoo` dans `lm()` : **le premier niveau est la référence**. Aucun message ne le rappelle.

Le workflow DESeq2

```
# 1. Vérifier que les échantillons sont dans le même ordre
identical(colnames(comptes), infos$Run)
#> [1] TRUE
infos_df <- infos |> column_to_rownames("Run")

# 2. Construire l'objet : comptes + description des échantillons + plan d'expérience
dds <- DESeqDataSetFromMatrix(countData = comptes, colData = infos_df,
                              design = ~ cell + dex)

# 3. Retirer les gènes presque jamais exprimés
dds <- dds[rowSums(counts(dds)) >= 10, ]

# 4. Normalisation, estimation des dispersions, test pour chaque gène
dds <- DESeq(dds)
res <- results(dds)
```

`DESeq()` affiche ses étapes (`estimating size factors`, `estimating dispersions`, `fitting model and testing`) : ce sont des messages, pas des erreurs.

Lire l'en-tête des résultats

```
res
#> log2 fold change (MLE): dex trt vs untrt
#> Wald test p-value: dex trt vs untrt
#> DataFrame with 22369 rows and 6 columns
#>                  baseMean log2FoldChange     lfcSE       stat      pvalue        padj
#>                 <numeric>      <numeric> <numeric>  <numeric>   <numeric>   <numeric>
#> ENSG00000000003  708.5979     -0.3812271 0.1007023  -3.785684 1.53286e-04 1.28920e-03
#> ENSG00000000419  520.2963      0.2068404 0.1121077   1.845014 6.50354e-02 1.94930e-01
#> ENSG00000000457  237.1621      0.0379543 0.1428231   0.265744 7.90437e-01 9.09900e-01
#> ENSG00000000460   57.9324     -0.0885309 0.2849344  -0.310706 7.56024e-01 8.92994e-01
#> ENSG00000000971 5817.3108      0.4264246 0.0888056   4.801775 1.57266e-06 2.06391e-05
#> ...                   ...            ...       ...        ...         ...         ...
#> ENSG00000273483   2.68955       0.849208  1.253365  0.6775423    0.498062          NA
#> ENSG00000273485   1.28646      -0.123613  1.588251 -0.0778298    0.937963          NA
#> ENSG00000273486  15.45244      -0.150429  0.482098 -0.3120292    0.755018    0.892519
#> ENSG00000273487   8.16327       1.045639  0.693057  1.5087349    0.131367    0.323298
#> ENSG00000273488   8.58437       0.108946  0.632300  0.1723006    0.863201    0.943415
```

Lire l'en-tête des résultats

- **`log2 fold change (MLE): dex trt vs untrt`** : traité **par rapport au** non traité. Un log2FC de 2 = 4 fois plus exprimé sous dexaméthasone. **Première ligne à lire**
- `baseMean` : expression moyenne normalisée
- `log2FoldChange`, `lfcSE` : l'effet et son incertitude
- `pvalue` : la p-value **brute** du gène
- `padj` : la p-value **ajustée** (FDR, Benjamini-Hochberg) : **celle qu'on utilise**

> Si l'en-tête affichait `dex untrt vs trt`, chaque gène « induit » serait en réalité « réprimé ». C'est la vérification la plus rapide d'une analyse DESeq2, et elle se fait **en lisant**.

Combien de gènes différentiellement exprimés ?

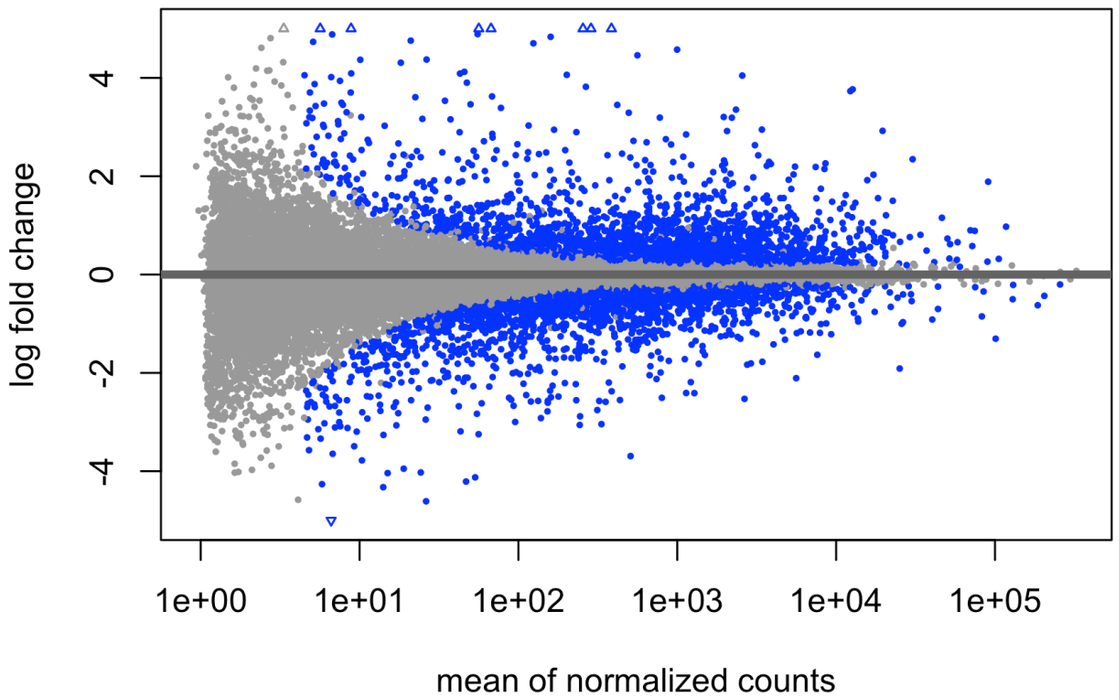
```r
sum(res$padj < 0.05, na.rm = TRUE)                          # FDR < 5 %
#> [1] 4000
sum(res$padj < 0.05 & abs(res$log2FoldChange) > 1, na.rm = TRUE)    # et au moins 2 fois
#> [1] 1018
sum(is.na(res$padj))
#> [1] 4337
```

Pourquoi des NA dans `padj` ? DESeq2 **ne teste pas** les gènes trop faiblement exprimés (filtrage indépendant) ni ceux qui ont une valeur aberrante : moins de tests, donc une correction moins sévère pour les autres.

> `sum(res$padj < 0.05)` sans `na.rm = TRUE` renvoie `NA` : c'est visible. Mais `df[df$padj < 0.05, ]` en R de base **garde les lignes NA** sans rien dire (module 5). Utilisez `filter()`, ou `which()`.

Le MA-plot

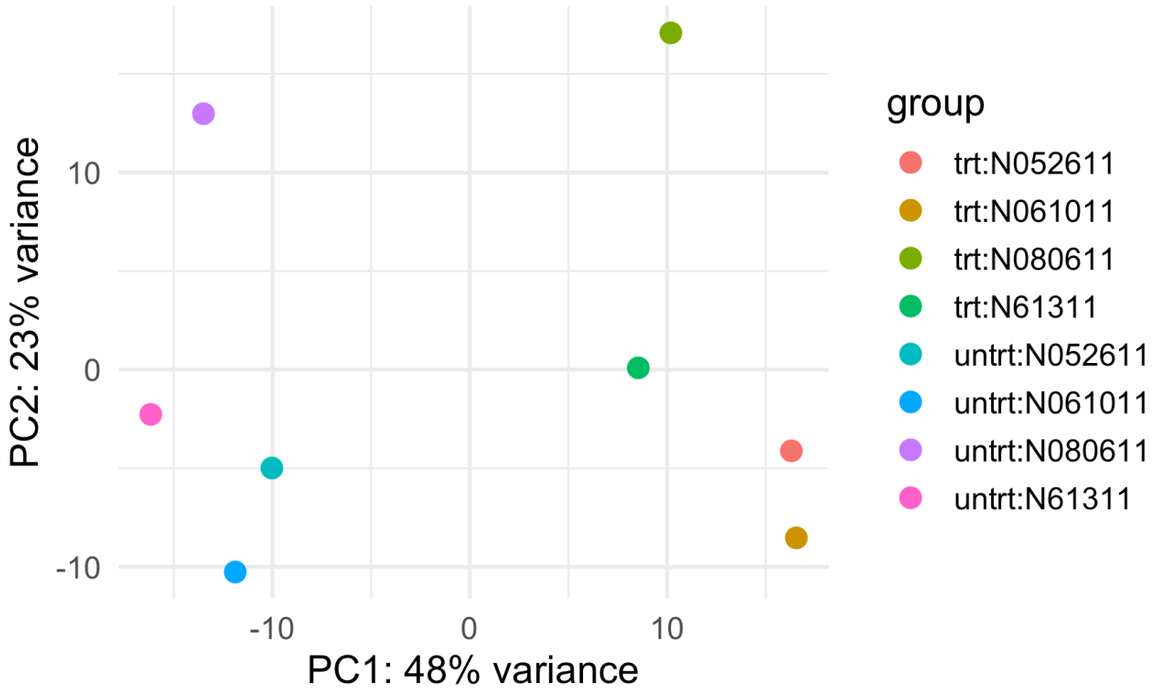
```
plotMA(res, ylim = c(-5, 5))
```



- **x** : expression moyenne (échelle log)
- **y** : log2 *fold change*
- en bleu : gènes avec `padj` < 0.1
- nuage centré sur 0 : la plupart des gènes ne changent pas, la normalisation a fonctionné
- dispersion plus forte à gauche : les gènes peu exprimés ont des log2FC très variables, rarement significatifs

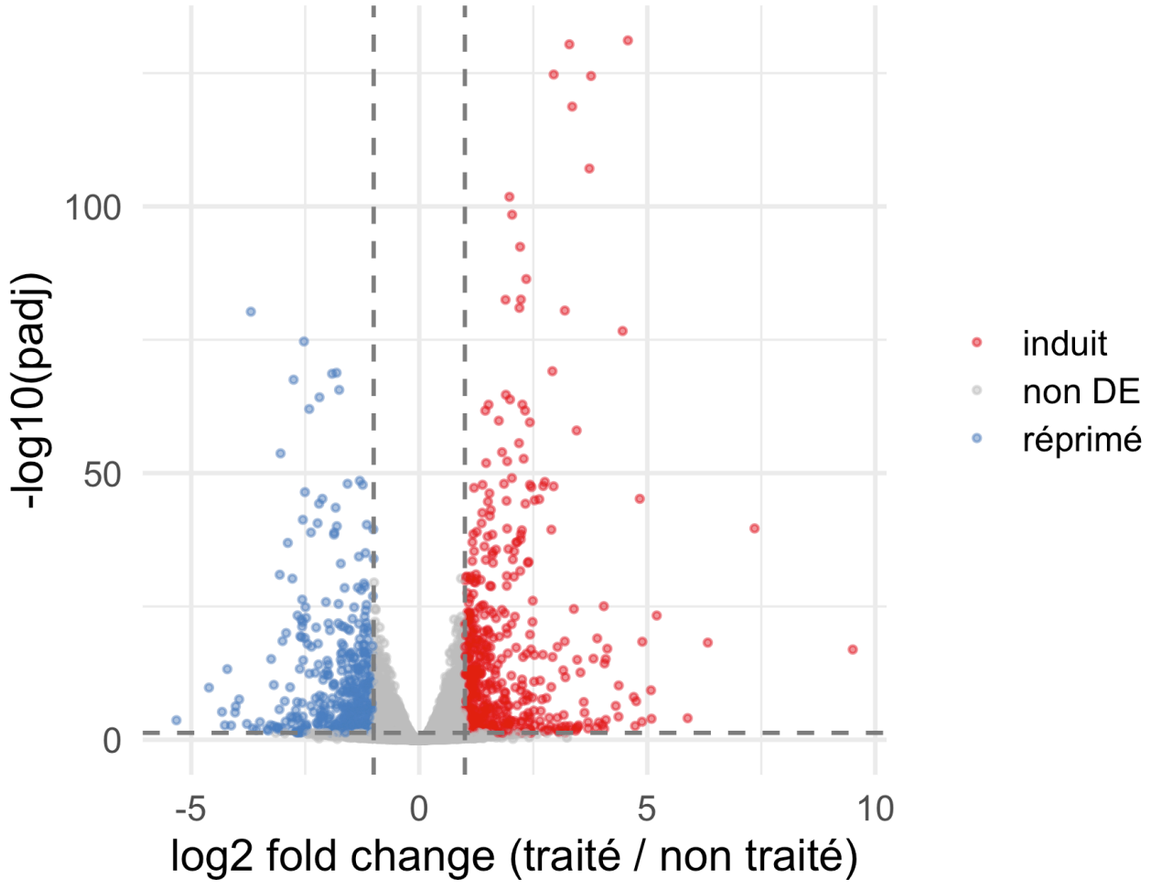
L'ACP : les échantillons se ressemblent-ils ?

```
vsd <- vst(dds, blind = FALSE)
plotPCA(vsd, intgroup = c("dex", "cell")) +
  theme_minimal(base_size = 14)
```



- `vst()` : transformation qui stabilise la variance, pour les graphiques (pas pour le test)
- **PC1** sépare les traités des non-traités : l'effet principal
- **PC2** sépare les lignées : un effet donneur, d'où `~ cell + dex`
- L'ACP sert de **contrôle qualité** : un échantillon isolé, des groupes mélangés, un effet de lot ?

Le *volcano plot*




```
res_tbl <- as.data.frame(res) |>
  rownames_to_column("gene") |>
  filter(!is.na(padj)) |>
  mutate(statut = case_when(
    padj < 0.05 & log2FoldChange > 1
      ~ "induit",
    padj < 0.05 & log2FoldChange < -1
      ~ "réprimé",
    .default = "non DE"))

ggplot(res_tbl, aes(x = log2FoldChange,
                    y = -log10(padj),
                    colour = statut)) +
  geom_point()
```

- à droite : **induits** par le traitement ; à gauche : **réprimés**
- en haut : les plus significatifs
- l'effet (x) et la significativité (y) sur un même graphique

✋ À vous : lire un résultat DESeq2

Une collègue vous montre ce résultat :

```
#> log2 fold change (MLE): dex untrt vs trt
#> Wald test p-value: dex untrt vs trt
#> DataFrame with 2 rows and 6 columns
#>                   baseMean log2FoldChange     lfcSE      stat       pvalue         padj
#>                  <numeric>      <numeric> <numeric> <numeric>    <numeric>    <numeric>
#> ENSG00000152583 997.44473       -4.57497  0.184241 -24.83137 4.11067e-136 7.41235e-132
#> ENSG00000002079   1.42773        1.97816  1.823348   1.08491  2.77963e-01           NA
```

1. Le gène `ENSG00000152583` (*SPARCL1*) est-il induit ou réprimé par la dexaméthasone ? De combien ?
2. Que signifie le `NA` de `padj` pour le second gène ? Est-il « non différentiellement exprimé » ?
3. Elle a compté les gènes DE avec `nrow(res_df[res_df$padj < 0.05, ])`. Que risque-t-elle ?

1. **Induit** : l'en-tête dit `untrt vs trt`, le non-traité **par rapport au** traité. -4.57 se lit donc 4.57 dans le sens traité / non traité : environ $2^{4.6} \approx$ **24 fois** plus exprimé sous dexaméthasone
2. **Non testé** : trop faiblement exprimé. On ne peut rien conclure, ni dans un sens ni dans l'autre
3. Les lignes `NA` sont **gardées** par le crochet : le compte est **gonflé**, sans message. `filter(padj < 0.05)` ou `which()`

TP3

Ressources

- *Modern Statistics for Modern Biology* (S. Holmes et W. Huber) : https://www.huber.embl.de/msmb/, chapitres 6 (tests) et 8 (RNA-seq)
- *RNA-seq workflow: gene-level exploratory analysis and differential expression* (M. Love, S. Anders, W. Huber) : https://bioconductor.org/packages/rnaseqGene/, le tutoriel de référence sur `airway`
- Vignette de DESeq2 : `vignette("DESeq2")`
- *Points of Significance*, chroniques de *Nature Methods* (2013-2020) : une notion statistique en deux pages
- *Statistics Done Wrong* (A. Reinhart) : https://www.statisticsdonewrong.com, les erreurs classiques expliquées simplement

Données : `penguins` (Gorman *et al.*, 2014 ; package `palmerpenguins`, CC0), `airway` (Himes *et al.*, 2014), `sleep` (Cushny et Peebles, 1905, repris par W. S. Gosset, dit « Student », 1908), `anscombe` (Anscombe, 1973).